



POLITECNICO
MILANO 1863

Requirement Analysis and Specification Document

Best Bike Paths Project

2025-2026

Deliverable:	RASD
Title:	Requirement Analysis and Specification Document
Authors:	Andrea Ferritti, Jieke Lin, Tommaso Tosca
Version:	1.1
Date:	25-January-2025
Download page:	https://github.com/ferritti/FerrittiLinTosca
Copyright:	Copyright © 2025, A. Ferritti, J. Lin, T. Tosca – All rights reserved

Contents

Table of Contents	3
List of Figures	5
List of Tables	5
1 Document Information	7
2 Introduction	8
2.1 Purpose	8
2.2 Scope	8
2.2.1 World Phenomena	9
2.2.2 Shared Phenomena	9
2.3 Definitions, Acronyms, Abbreviations	10
2.3.1 Definitions	10
2.3.2 Acronyms	11
2.3.3 Abbreviations	11
2.4 Revision history	11
2.5 Reference Documents	11
2.6 Document Structure	11
3 Overall Description	12
3.1 Product perspective	12
3.1.1 Scenarios	12
3.1.2 Domain Class Diagram	14
3.1.3 State Diagrams	15
3.2 Product functions	17
3.3 User characteristics	18
3.3.1 Guests	18
3.3.2 Users	18
3.4 Assumptions, dependencies and constraints	19
3.4.1 Domain Assumptions	19
3.4.2 Dependencies	19
4 Specific Requirements	20
4.1 External Interface Requirements	20
4.2 Functional Requirements	27
4.2.1 Use Cases Diagrams	29
4.2.2 Use Cases	31
4.2.3 Sequence Diagrams	40
4.2.4 Requirements Mapping	56
4.3 Performance Requirements	62
4.4 Design Constraints	62
4.4.1 Standards Compliance	62
4.4.2 Hardware Limitations	62
4.5 Software System Attributes	63
4.5.1 Reliability & Availability	63
4.5.2 Security	63
4.5.3 Maintainability	63
4.5.4 Portability	63

5	Formal Analysis Using Alloy	64
5.1	Signatures	64
5.2	Facts	65
5.3	Assertions	66
5.4	Model Visualization	67
6	Use of Generative AI	70
6.1	AI-Based Support Tools	70
6.2	Example of Interaction: Mockup Generation	70
6.3	Example of Interaction: Rules Identification	72
6.4	Conclusion	73
7	Effort Spent	74

List of Figures

1	Domain Class Diagram	14
2	Trip Lifecycle State Diagram	15
3	Path Lifecycle State Diagram	16
4	User Profile Lifecycle State Diagram	17
5	Home Page	20
6	Start navigation screen	21
7	Recent trips screen	26
8	General Utilities Use Case Diagram	29
9	Manual Mode Use Case Diagram	30
10	Automated Mode Use Case Diagram	30
11	[SD1]: Sign Up Sequence Diagram	42
12	[SD2]: Login Sequence Diagram	43
13	[SD3]: Search Path Sequence Diagram	44
14	[SD4]: Start Navigation Sequence Diagram	45
15	[SD5]: Save Trip Sequence Diagram	46
16	[SD6]: View Trip Sequence Diagram	47
17	[SD7]: Edit Profile Sequence Diagram	48
18	[SD8]: Delete Path Sequence Diagram	49
19	[SD9]: Publish Path Sequence Diagram	50
20	[SD10]: Draw Path Sequence Diagram	51
21	[SD11]: Add Obstacle Sequence Diagram	52
22	[SD12]: Perform Automated Path Acquisition Sequence Diagram	53
23	[SD13]: Stop Path Acquisition Sequence Diagram	54
24	[SD14]: Publish Recorded Path Sequence Diagram	55

List of Tables

1	[UC-01]: Sign Up Use Case	31
2	[UC-02]: Login Use Case	32
3	[UC-03]: Search Path Use Case	32
4	[UC-04]: Start Navigation Use Case	33
5	[UC-05]: Save Trip Use Case	34
6	[UC-06]: View Trips Use Case	35
7	[UC-07]: Edit Profile Use Case	35
8	[UC-08] Delete Path Use Case	36
9	[UC-09]: Publish Path Use Case	37
10	[UC-10]: Draw Path Use Path	37
11	[UC-11]: Add Obstacle Use Case	38
12	[UC-12]: Perform Automated Path Acquisition	38
13	[UC-13]: Stop Path Acquisition Use Case	39
14	[UC-14]: Publish Recorded Path Use Case	39
15	Goal 1 mapping	56
16	Goal 2 mapping	57
17	Goal 3 mapping	58
18	Goal 4 mapping	59
19	Goal 5 mapping	60
20	Goal 6 mapping	60
21	Goals Mapping Table	61

22	Effort - Andrea Ferritti	74
23	Effort - Jieke Lin	74
24	Effort - Tommaso Tosca	74

1 Document Information

Version	Modification Details
v1.0 23-12-2025	First version.
v1.1 25-01-2025	Modifications: • Added a loop in SD4 enclosing the navigation interaction sequence until the destination is reached; • Refactored SD5 to start directly with showPage(TripSummary); • Refactored SD4 to include Weather and Map Services, ensuring consistency with corresponding Use Cases; • Removed ‘modify obstacle’ from SD5 and UC5, as the summary already includes the mini-map; • Removed the loop in SD6, as all user trips are retrieved from the database at once instead of iteratively.

2 Introduction

2.1 Purpose

The growing popularity of cycling as both a means of transportation and a recreational activity has increased the need for reliable, up-to-date information about bike paths. Cyclists often struggle to identify safe and efficient paths, especially when facing incomplete or outdated data regarding path conditions, obstacles, or environmental factors. At the same time, users who wish to contribute to the cycling community by sharing information about the paths they travel may find it difficult to organize, store, and communicate such data in a structured and meaningful way. The purpose of BBP is to provide an environment where cyclists can record their trips, contribute information about bike paths, and access curated, community-driven data to make informed decisions about their paths. Through a combination of manual input and automated data collection from mobile devices, users can keep track of their cycling activities while simultaneously improving the shared knowledge base about path conditions. BBP aims to give all users, whether registered or not, a clear understanding of the quality and safety of available bike paths, offering path visualizations, condition assessments, and recommendations based on consolidated and verified information. Transparency, reliability, and community collaboration are promoted at every step, ensuring that cyclists can navigate efficiently while benefiting from accurate and continuously updated path information.

BBP, therefore, intends to serve the following primary goals:

- [G1] Registered users would like to maintain a history of their trips.
- [G2] Registered users would like to access performance metrics about their trips.
- [G3] When available, registered users would like to be informed about meteorological information related to their trips.
- [G4] Registered users would like to share information about specific bike paths.
- [G5] Users would like to find the optimal bike path between an origin and a destination by comparing viable alternatives based on efficiency and status.
- [G6] Users would like to access reliable and up-to-date shared information about bike paths.

2.2 Scope

The Best Bike Paths (BBP) system aims to support the biking community by mediating the interactions between individual cyclists and the shared repository of information regarding bike paths. The external users who interact with the platform include registered cyclists, unregistered visitors, and potentially local cycling associations or municipalities interested in monitoring the conditions of bike paths. Since only registered users can contribute new information or validate automatically collected data, the system must identify these users through personal accounts, while unregistered visitors access the platform in a read-only mode. Registered cyclists provide BBP with details about their trips either manually or through an automated acquisition mode that collects GPS data and movement information. In addition to recording the path itself, the application also provides users with performance metrics derived from their trips, such as total distance, duration, average speed, and other relevant indicators to support personal activity monitoring and analysis. These data sources become key assets for enhancing trip analysis, improving the accuracy of anomaly detection, and increasing the overall reliability of the platform's scoring mechanisms. When available, the system also enriches trip data with meteorological information, such as weather conditions, temperature, and wind speed, retrieved from external services. The system supports users in reviewing, correcting, and publishing the collected information, and it aggregates contributions

from multiple cyclists to ensure that the status of each path reflects fresh and trustworthy evidence. All external users can request the visualization of one or more paths between an origin and a destination, and BBP assists them by computing ranked paths based on their status and effectiveness. Through these features, the application facilitates the discovery of safe and efficient bike paths and provides a continuously updated overview of the cycling network.

2.2.1 World Phenomena

- [WP1] The registered user wants to keep track of his trips.
- [WP2] The registered user wants to share a path to other people.
- [WP3] The cyclist wants to find the best path between two locations.
- [WP4] The cyclist moves along a path.
- [WP5] The cyclist's physical speed varies over time.
- [WP6] The cyclist's device experiences vibrations or impacts due to road conditions.
- [WP7] The cyclist reaches the starting location of the trip.
- [WP8] The cyclist arrives at the ending location of the trip.
- [WP9] The cyclist trip is conditioned by the weather conditions.
- [WP10] The cyclist wants to interrupt the current trip.

2.2.2 Shared Phenomena

World Controlled

- [SPW1] The registered user publishes information about the path in manual mode.
- [SPW2] The registered user accepts path information suggested by automated mode.
- [SPW2] The registered user edits path information suggested by automated mode.
- [SPW3] The system receives accelerometer readings generated by vibrations.
- [SPW4] The system receives gyroscope readings generated by device rotation.
- [SPW5] The system receives GPS location data from the cyclist's device.
- [SPW6] The registered user save a trip.
- [SPW7] The cyclist inserts an origin and a destination.
- [SPW8] The cyclist starts the navigation.
- [SPW9] The registered user edits profile information.

Machine Controlled

- [SPM1] The system suggests path information to be inserted in automated mode.
- [SPM2] The system provides the top 5 paths between two locations based on the score.
- [SPM3] The system provides information about the path of interest.
- [SPM4] The system provides performance metrics about the user's trip.
- [SPM5] The system show all user's trips.
- [SPM6] The system show all the published paths of the user.
- [SPM7] The system show the main user statistics.
- [SPM8] The system collects user path reports.
- [SPM9] The system updates the status of a path based on the reports.

2.3 Definitions, Acronyms, Abbreviations

2.3.1 Definitions

- **Bike Path:** A road segment or sequence of segments that is suitable for bicycles. A path is considered bike-safe if it includes a dedicated bike lane or if car traffic is rare and speed limits are compatible with typical bike speeds.
- **Trip:** A cycling activity recorded by a registered user. A trip includes GPS tracking data, performance metrics (distance, average speed, etc.), and, when available, meteorological information retrieved from an external weather service.
- **Performance Metrics:** Statistics associated with a trip, such as total distance, duration, average speed, maximum speed and other cycling-related measures.
- **Meteorological Information:** External data enriched by BBP when available, including temperature, weather conditions (sunny, cloudy, rainy...), wind speed, humidity, and similar environmental indicators.
- **Manual Mode:** A mode in which a user manually enters information about a bike path, specifying street names, path segments, their status, and any obstacles.
- **Automated Mode:** A mode in which BBP automatically collects data from the user's mobile device while biking. Data includes GPS traces to infer the following path, accelerometer and gyroscope readings to detect potential potholes or obstacles, and speed information to infer that the user is cycling.
- **Obstacle:** Any relevant physical irregularity affecting the bike path (e.g., potholes, bumps, debris). In automated mode, obstacles are inferred from device sensors and later confirmed by the user.
- **Status:** A qualitative assessment of a bike path segment. Possible values include: optimal, medium, sufficient, requires maintenance, etc. Status can be user-generated (manual or automatic) and may be aggregated from multiple users.
- **Information Publishable:** Verified and user-approved data about a bike path (status, obstacles) that can be made publicly accessible to the community.
- **Score:** A computed score used to rank alternative bike paths between an origin and a destination. The score considers both the quality (status) of the path and its effectiveness in connecting the two points.

2.3.2 Acronyms

- **BBP:** Best Bike Path.
- **GPS:** Global Positioning System.

2.3.3 Abbreviations

- **G:** Goal.
- **SP:** Shared Phenomena.
- **WP:** World Phenomena.

2.4 Revision history

- Version 1.0: 23/12/2025
- Version 1.1: 25/01/2025

2.5 Reference Documents

- Assignment RDD AY 2025-2026
- Software Engineering 2 AY 2025/2026 slides "CreatingRASD"

2.6 Document Structure

1. **Introduction:** It aims to provide a brief overview of the project, focusing in particular on the motivations behind it and the goals intended to be achieved through its development.
2. **Overall Description:** It is a high-level description of how the system operates, including a detailed explanation of the phenomena involving the world, the machine, or both. This section also provides the domain description along with the related assumptions.
3. **Specific Requirements:** This section presents a detailed analysis of the requirements needed to achieve the identified goals. In addition, it provides further information relevant to developers, such as hardware and software constraints.
4. **Formal analysis:** It provides a formal specification of the world phenomena using Alloy.
5. **Use of Generative AI:** In this chapter, examples of interactions are provided to explain how AI was used to support the drafting of the document.
6. **Effort spent:** It reports the time spent to produce this document, organized by section.
7. **References:** It includes references to all documents and software used in the preparation of this paper.

3 Overall Description

3.1 Product perspective

3.1.1 Scenarios

Users (authenticated)

User publishes path information in automated mode

After completing the bike ride, user U clicks the stop button related to the automated mode and will be redirected to the path data entry page with all the field pre-filled by the system based on the data it collected, with the possibility to modify the status and the obstacles. Finally, U presses the "Publish" button, and the new path becomes immediately visible to all users.

User publishes path information in manual mode

User U wants to share a path they frequently use with the community. From the main map interface, U presses the "Create Path" button (the plus icon). U enters the drawing mode and taps a sequence of points on the map to define the itinerary; the application automatically interprets these inputs and snaps the route line to the correct street geometry. If U is aware of specific obstacles along this route, such as deep potholes or dangerous intersections, U switches the tool to "Obstacles" mode and taps on the corresponding points of the path to flag them. Once the geometry is defined, U proceeds to the next step. The application presents a summary form where U enters the estimated duration and selects a status rating. The system automatically lists the names of the streets detected along the drawn path. Finally, U presses the "Publish" button, and the new path becomes immediately visible to all users.

User eliminates a published path

User U wants to delete a previously published route. In the navbar, U clicks on "Published," where he will find all the paths they have published. U clicks on the route they want to delete; the page for editing the route's information will open, and they click on the icon associated with deleting the path.

User modifies personal information

User U wants to edit their personal information. In the navbar, U clicks on "Profile" and then on "Settings", where he will find all their personal information with the option to modify it

User discards the collected path information

User U wants to discard the path information he entered. On the data entry screen for the given route, U clicks the "Discard" button, and the information will not be published to the community. However, if the information was entered in automated mode, it will still be added to the recent trips.

User saves the trip

User U wants to save the completed trip. Once the trip is finished, the end-of-path screen appears, showing the distance traveled and the average speed, along with the option to submit a report on the path's status and obstacles. Finally, U clicks the "Save trip" button and the trips is saved into Recent Trips.

User inserts an already existing path

User u enters the information for a new path, but the streets in the entered route are the same as those of an existing path. In this case, the application will notify the user with a warning that the entered streets correspond to those of an already existing path. The same thing happens whether the insertion is done manually or automatically

Guests (unauthenticated)

Guest registers in the application

Guest G, having navigated to the Login screen and clicks the "Registration" link presented on this page, is immediately redirected to the dedicated registration page. Here, G is required to complete the registration form by filling in all the necessary information: they enter a valid email address and define a secure password. After reviewing the entered data, G presses the final "Register" button. A verification email will then be sent to the provided email address, containing a verification code that has to be inserted in the application in the appropriate textbox, to confirm that the entered email address exists. Upon successful completion of the registration process, the system confirms the creation of the new account and redirects G back to the main Login screen, prompting him to perform their first access.

Guest logs into the application

Guest G, currently viewing the Home Page in the unregistered state, clicks on "My Profile" in the navbar. This action directs G to the Login screen. On this screen, G is prompted to enter their credentials: they fill in the email address in the designated field and the corresponding password in the password field. After ensuring the data is correct, G clicks on the "Login" button. Upon successful validation, the system grants G access and redirects him to the profile page.

Cyclists (users and guests)

Cyclist searches for a path

Cyclist C wants to find the best path to their destination. From the main map interface, C enters the desired origin and destination points into the floating search bar. The system calculates and displays the five highest-scoring paths on the map. Each path is shown with its associated score, including the visualization of the corresponding obstacles present along the path. The path with the highest score is highlighted in teal to indicate the recommended route, while the other alternatives are displayed for comparison, allowing C to choose the best option before starting the ride.

Cyclist starts a navigation

After entering the origin and destination and selecting the desired path, cyclist C wants to start navigation. From the main map interface, C clicks the "Start Navigation" button, and the navigation screen opens. On the map, a cursor representing C's current position is displayed; at the top, the current speed and street name are shown, while at the bottom there is an "Exit Navigation" button to end the navigation.

3.1.2 Domain Class Diagram

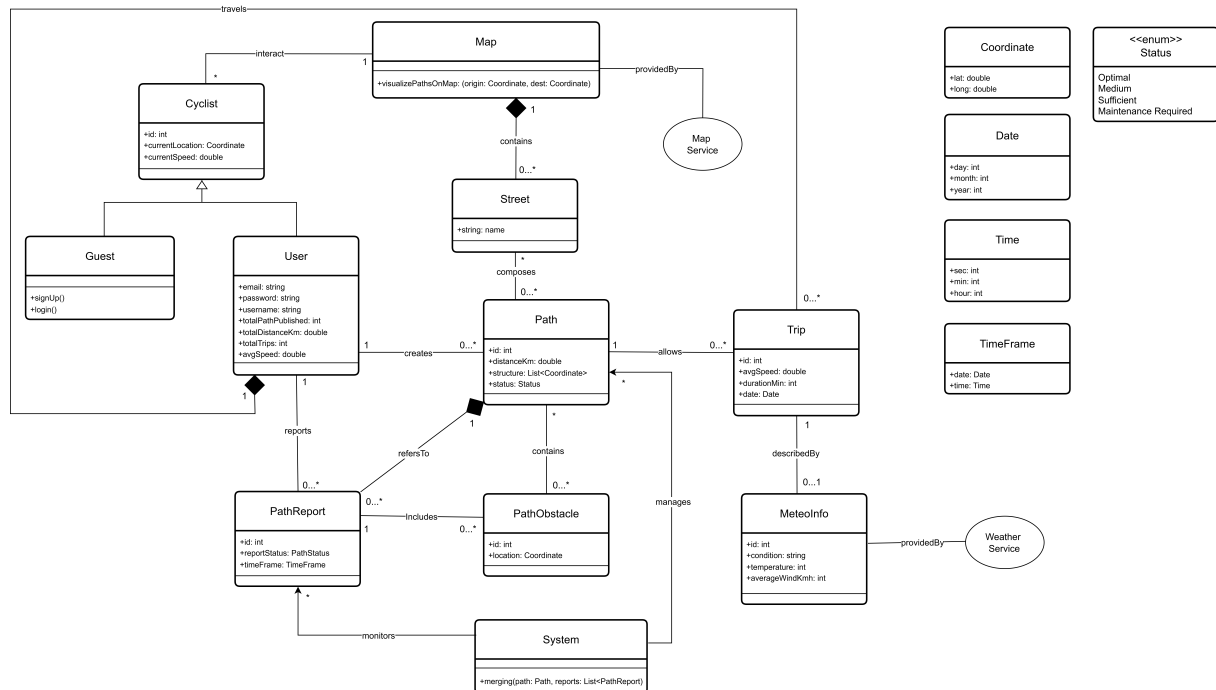


Figure 1: Domain Class Diagram

Notes:

- The *creates* relationship indicates the publication of the *Path*, while *reports* tracks the contributions regarding obstacles and status, which is then aggregated by the *System* (Merging).
- The *status* attribute in *Path* is initially set by the user who published the path, and is later derived from the aggregation of the most recent *PathReport*.
- The *structures* attribute in *Path* is a list of points in the map that allows the *System* to draw the path. In manual mode they are manually inserted on the map by the user who publishes the path, while in automated mode they are automatically inserted by the *System*.

3.1.3 State Diagrams

Trip Lifecycle

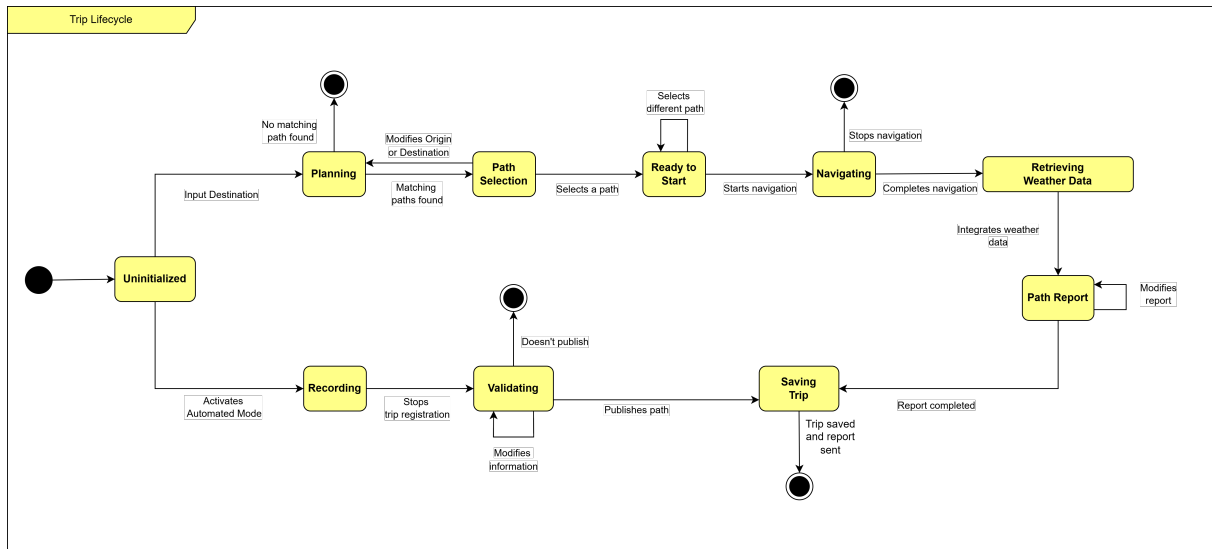


Figure 2: Trip Lifecycle State Diagram

The Trip Lifecycle State Diagram illustrates the two operational modes managed by the system. In the upper branch, dedicated to assisted navigation, the process begins with the user entering a destination. While the system automatically detects the current position as the default origin, the workflow is designed to be flexible: as shown by the feedback transition from Path Selection to Planning, the user can subsequently modify both the origin and the destination if the calculated paths are unsatisfactory, triggering a new planning phase. Once a route is confirmed, the system transitions to Ready to Start and then to Navigating. Upon completion, the lifecycle enters the Retrieving Weather Data state to integrate meteorological information, followed by the Path Report state, where the user can review and modify the report details. In parallel, the lower branch describes the automated mode, designed for passive tracking. Here, the trip immediately enters the Recording phase from the uninitialized state. Once recording is stopped, the process moves to Validating, where the acquired data can be reviewed. Both flows finally converge in the concluding Saving Trip state, ensuring that regardless of the input mode, all metrics, coordinates, weather data, and reports are correctly persisted in the system database.

Path Lifecycle

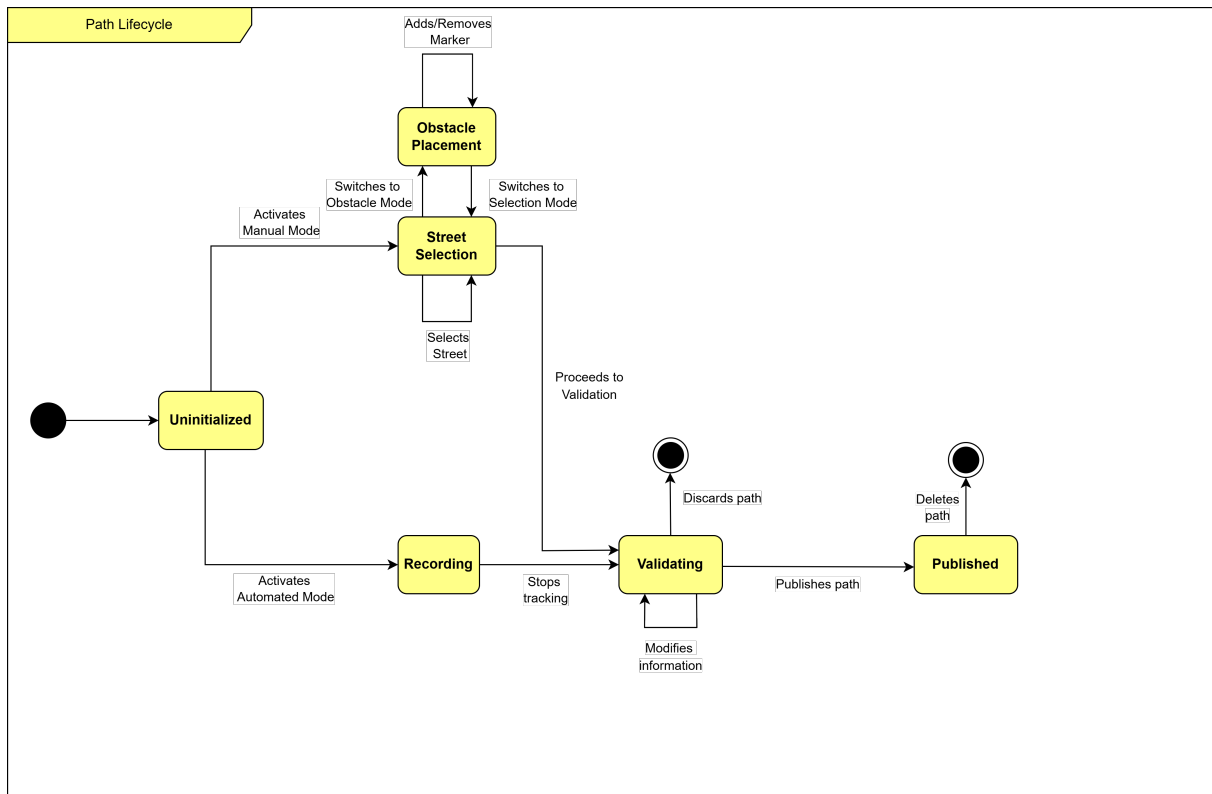


Figure 3: Path Lifecycle State Diagram

The Path Lifecycle State Diagram illustrates the process of creating and publishing a new path within the system. Starting from the initial state, the flow branches into two distinct operational modes. The manual mode allows the user to compose the path by interactively selecting existing streets of interest on the map and manually placing obstacle markers, with the ability to freely switch between these activities. Alternatively, the automated mode acquires the track via real-time GPS recording. Both flows converge in the validation phase; in this state, the user can review and eventually modify the collected information before final confirmation. The cycle concludes with the publication of the path, or with the discard of the path if not confirmed.

User Profile Lifecycle

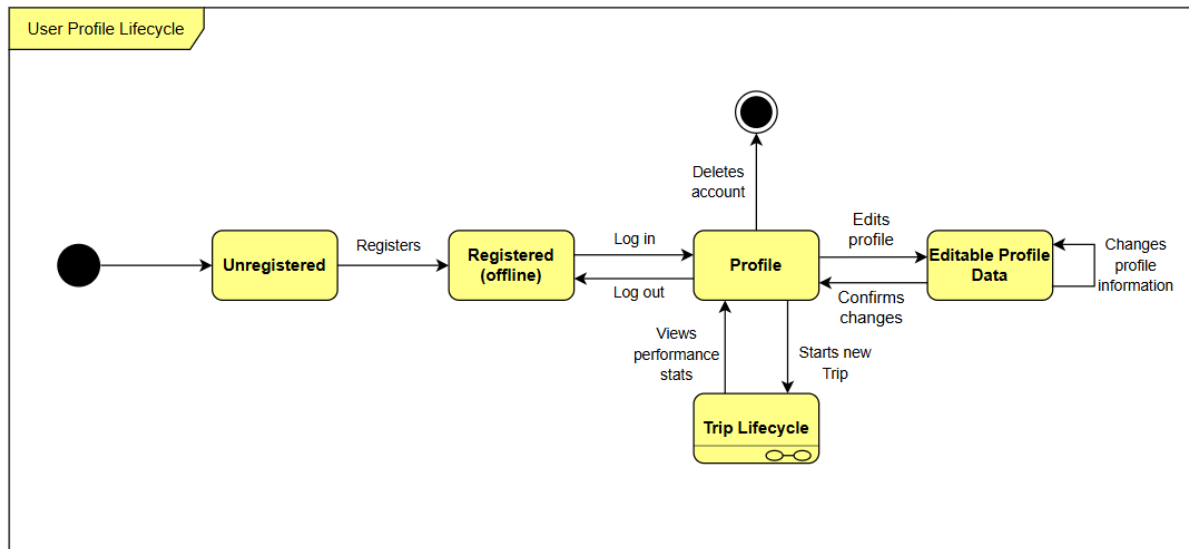


Figure 4: User Profile Lifecycle State Diagram

The lifecycle begins when a visitor without an account completes registration, creating a persistent profile in the system that starts in a signed-out state. Entering credentials activates the working session, allowing the user to modify personal data or undertake new trips to improve their performance statistics. Upon finishing these activities, logging out returns the user to an inactive state while keeping the updated data saved for future access. The cycle ends permanently only if the user chooses to delete their account, an action that irreversibly removes the profile from the system.

3.2 Product funtions

This section outlines the principal functionalities of the platform.

Sign up & Login

To sign up for the system, a user must provide his name, an email address, and a password. The system checks whether there is an existing account with the same email address: if so, the user is notified that the account already exists; otherwise, the registration proceeds and a verification code is sent to the provided email address. To complete the registration, the user must enter the verification code in the designated textbox. To log in, a user must enter the email and the associated password, and the system checks if the credentials match with an existing account: if so, the log in is completed; otherwise, the user is notified that the credentials are wrong.

Path search and visualization

The system allows users to find the best paths that connect two locations. They must enter the starting point and destination, and the system will display on the map the paths with the highest score, along with their related obstacles. It is also possible to access the information screen of the selected path.

Publication of a path in manual mode

The system allows registered users to publish new paths by manually providing the required information. The user draws the path and adds obstacles directly on the map, and then enters the remaining information.

Publication of a path in automated mode

The system allows registered users to publish new paths by providing the required information automatically. The system retrieves data from GPS and sensors (accelerometer, gyroscope) while the user is biking, and automatically fills in all the necessary information, and the user can choose to accept, modify, or discard it.

Navigation

After the user has selected the desired path from the map and starts navigation, the system provides a real-time representation of the user's position on the map and also displays the current speed and the street being traveled.

Trip tracking and analysis

The system allows registered users to keep track of their trips. After completing the trip, a screen will appear showing the trip statistics, and once the confirmation button is clicked, the system saves the trip and, if available, enriches it with meteorological information (weather, wind, temperature) retrieved from an external service.

Path reporting

The system allows registered users to submit a report regarding the status and the presence of obstacles on an existing path. After completing a trip associated with the path in question, the user can modify the status and add or remove any obstacles, and once the confirmation button is clicked, the report will be sent to the system.

Data Aggregation and Merging

The system collects path reports from multiple users and merges conflicting information. It calculates the current status of a path based on the freshness of the reports and the number of confirming data obtained. For the obstacles, it merges all the obstacles reported in all reports and adds them to the associated path. If an obstacle is removed in one report but kept in another, it will be retained.

3.3 User characteristics

3.3.1 Guests

Guests include casual users, tourists, or people who simply want to find a bike path without tracking their data. They can search for the best paths between two locations, select a desired path, and start the real-time navigation to follow the path, but without the ability to save the trip or view post-activity statistics.

3.3.2 Users

Users are able to login to the platform. Their profiles contain personal information and some statistics such as total kilometers travelled, total trips completed, total paths published and the average speed of their trips. In addition to all the features available to guests, they can publish new paths (in either manual or automated mode) and submit reports regarding existing paths. Furthermore, they can record their trips and save them to populate their history and view detailed statistics at the end of the navigation.

3.4 Assumptions, dependencies and constraints

3.4.1 Domain Assumptions

This section describes the fundamental assumptions regarding the environment and the behavior of all entities interacting with the system. These assumptions simplify the system's design and implementation by setting clear expectations for the users, and the platform itself, ensuring the system functions efficiently within its intended context

[D1] The user's device is equipped with a GPS, an accelerometer and a gyroscope.

[D2] The user has a working internet connection.

[D3] The user provides truthful information regarding the path to be published.

[D4] The user provides truthful reports regarding paths.

[D5] The external weather information service is reliable.

[D6] The external map service is available and provides accurate topographical data.

3.4.2 Dependencies

- **Map Service:** it's responsible for providing the map that enables the graphical visualization of the paths and all the related operations. This is a critical dependency: if this service is unavailable, the core functionalities of the system cannot be performed.
- **Weather Service:** it's responsible for adding weather information to the users' trips. This is a non-critical dependency: if this service is unavailable, the system will continue to function normally, simply omitting the weather information from the trip details.

4 Specific Requirements

4.1 External Interface Requirements

Here is shown the mockups of the UI. With the exception of the home page, which is shown from both the guest's and the user's perspectives, all other screens are displayed only from the user's perspective, since the only differences are already shown on the home page.

Home page

The *Home page* acts as a dashboard for exploration. It assumes the cyclist immediately wants to either find a safe path to ride or start recording their own journey. On the left, the home page is shown from the user's perspective; on the right, from the guest's perspective

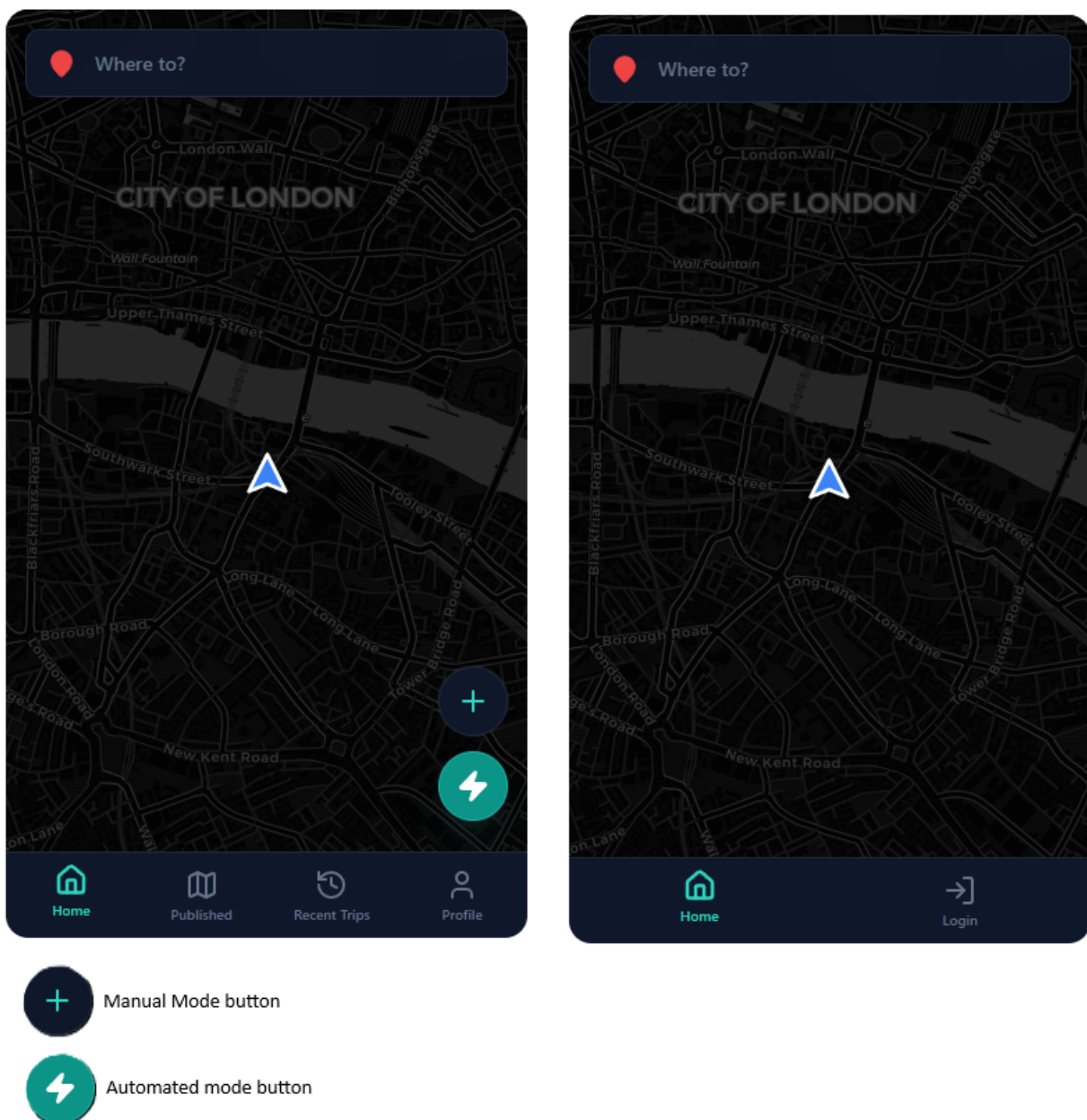


Figure 5: Home Page

Search path

From the *Home Page*, when the cyclist inserts a destination, the five paths that connect origin and destination with the highest score will appear on the map. If the cyclist wants to change the origin to a location that is not the current location, the “Start Navigation” button will disappear.

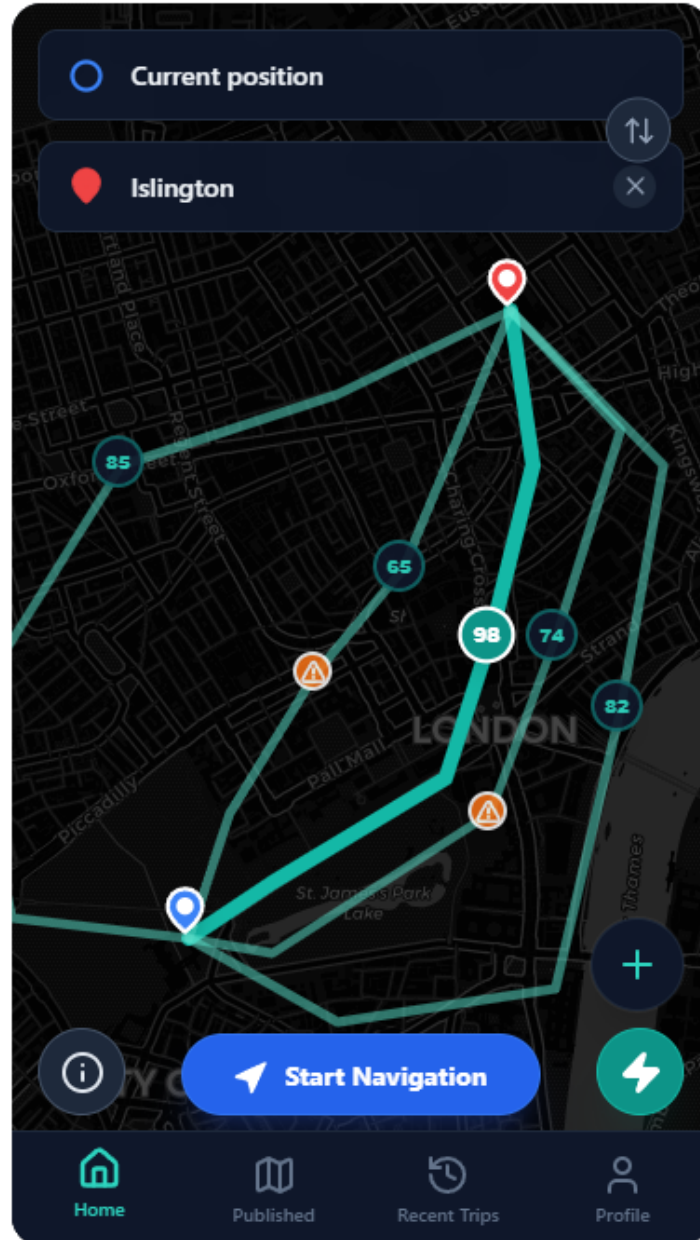
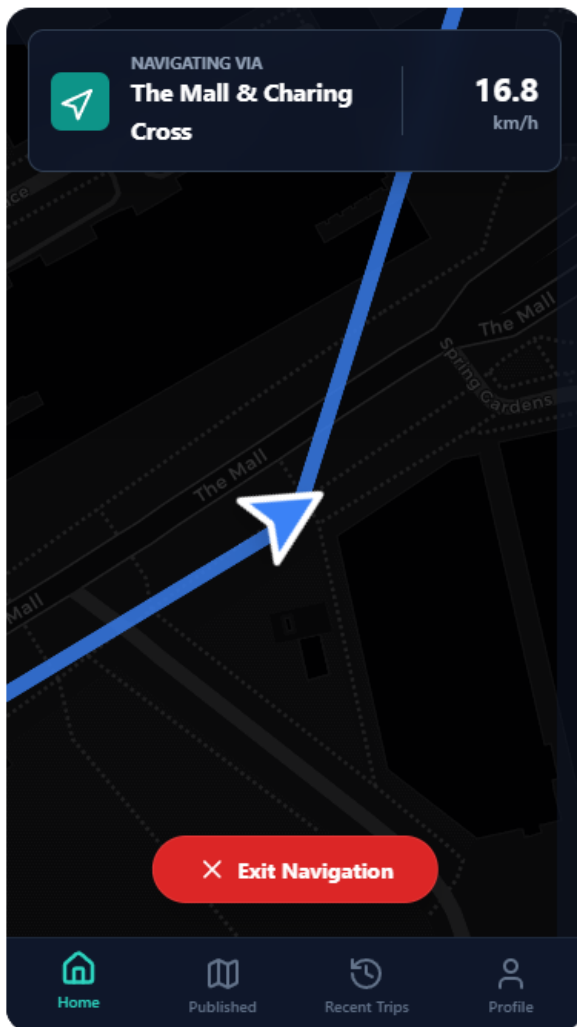


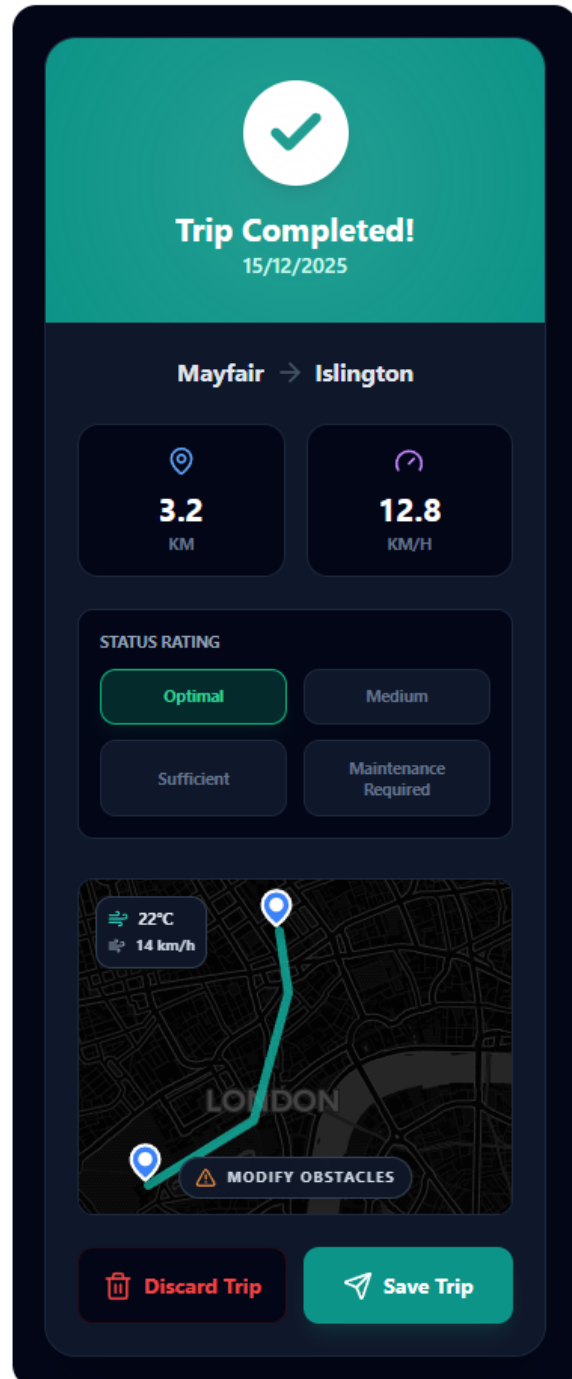
Figure 6: Start navigation screen

Navigation

From the *Search path screen*, when the cyclist clicks “Start Navigation”, the navigation begin and the *Navigation screen* appears. When the cyclist reaches the destination, the navigation ends and the *Trip completed screen* appears.



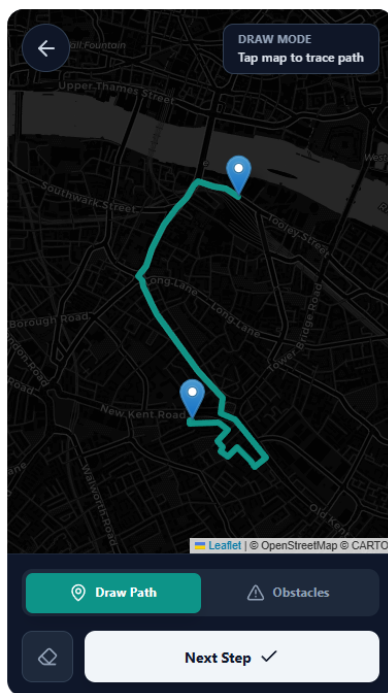
(a) Navigation screen



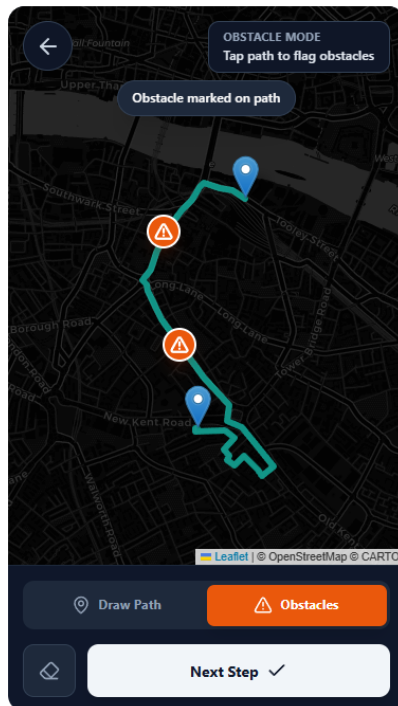
(b) Trip completed screen

Manual Mode

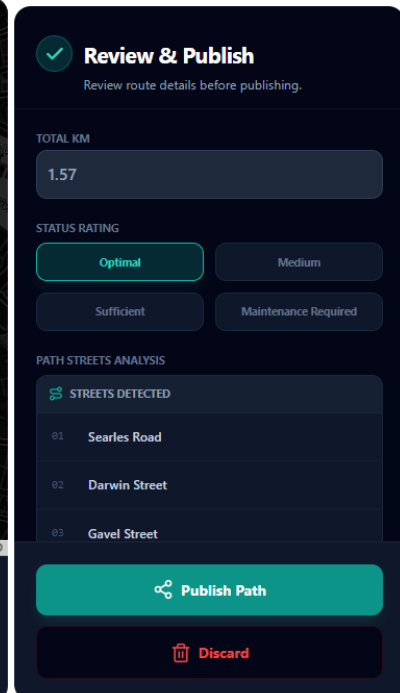
From the *Home page*, when the user clicks the *Manual Mode button*, the *Draw Path screen* appears. Here, the user can draw a path by clicking on the map to add points, which the `press()` then connects to create a path that follows the street layout. By clicking “Obstacles”, the *Insert Obstacles screen* appears, allowing the user to place obstacles directly on the map. Finally, clicking “Next Step” opens the *Review & Publish screen*, where the user sets the status and publishes the path by clicking “Publish Path”.



(a) Draw path screen



(b) Insert obstacles screen



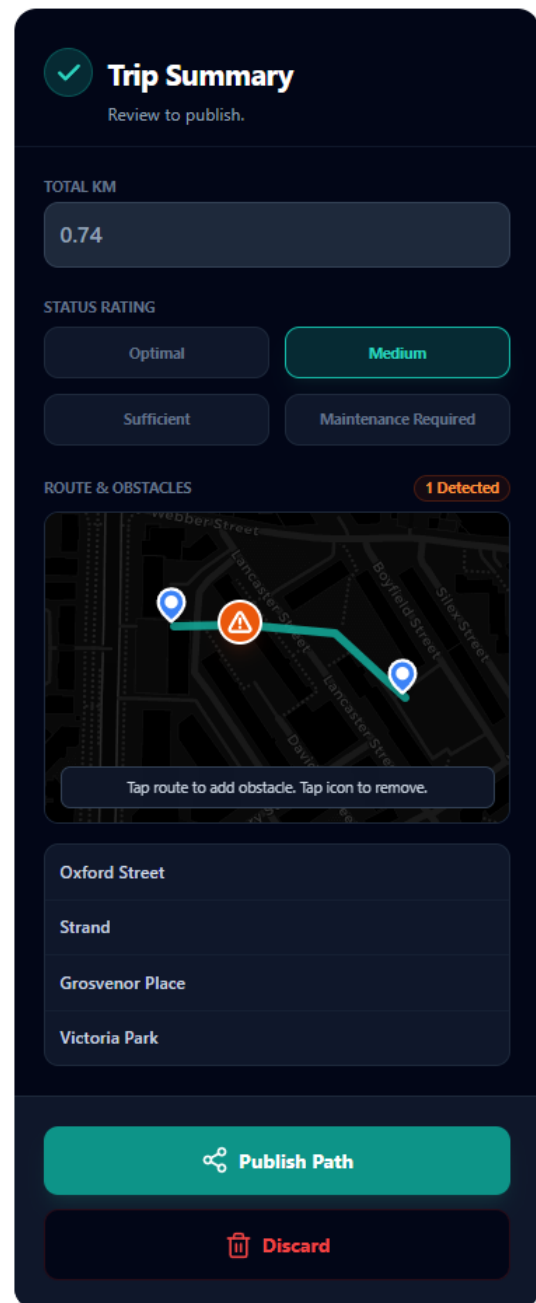
(c) Review & publish screen

Automated Mode

From the *Home page*, when the user clicks the *Automated Mode button*, the *Automated Mode screen* appears. Here, statistics such as current speed, average speed, and distance are displayed. Clicking the *Stop button* stops the recording and opens the *Trip Summary screen*, which shows the trip statistics recorded by the device sensors.



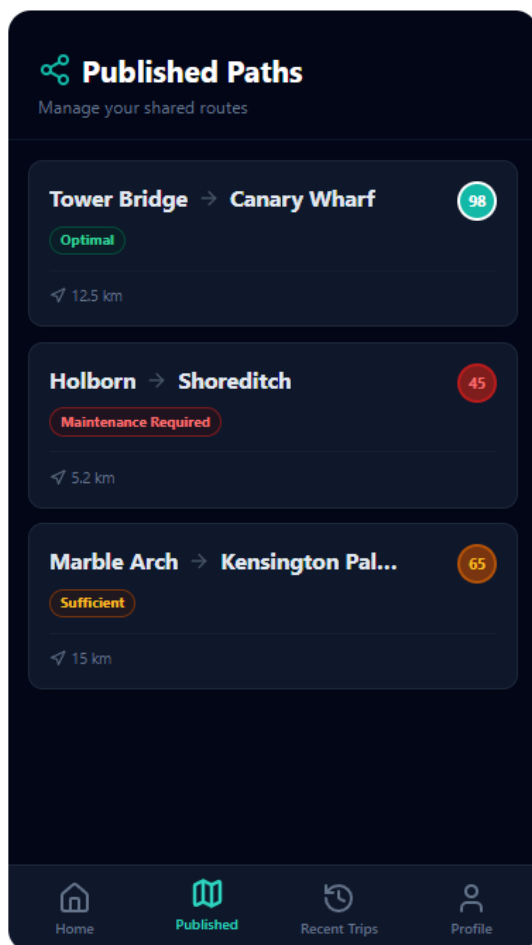
(a) Automated mode screen



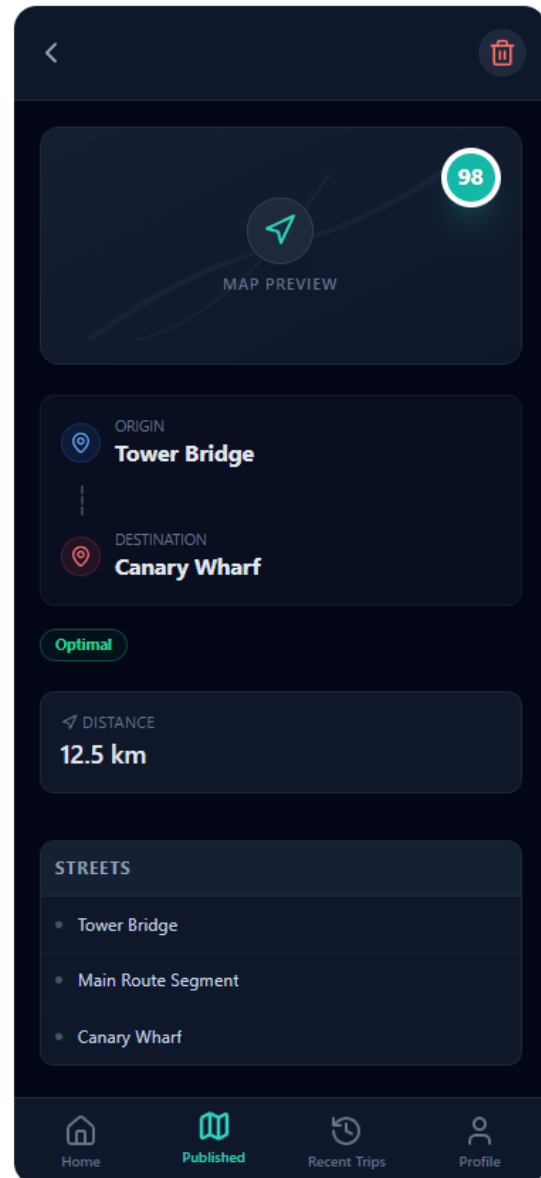
(b) Trip summary screen

Published paths

the *Published paths* screen shows all the paths published by the user. Selecting a path opens the *Path info* screen, which displays detailed information about the selected path.



(a) Published paths screen



(b) Path info screen

View trips

the *Recent trips* screen lists all the trips made by the user, sorted by date.

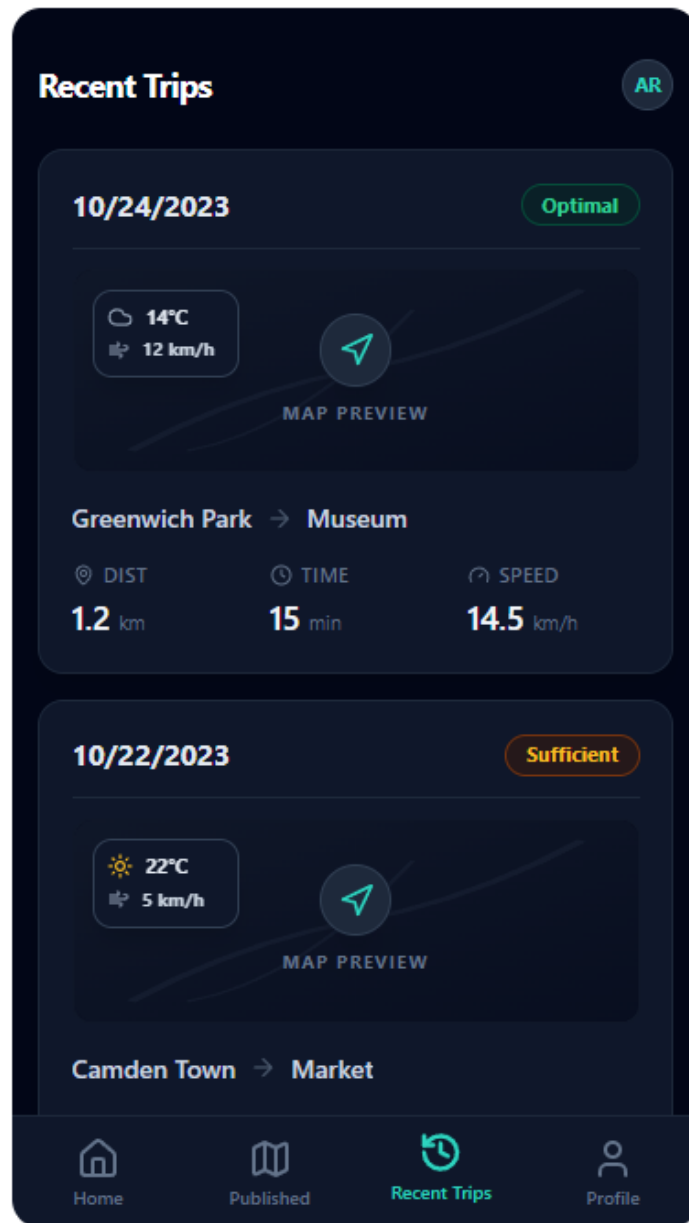


Figure 7: Recent trips screen

4.2 Functional Requirements

Sign up & Login

[R1] The platform shall allow new users to register an account by providing personal information.

[R2] The platform shall allow registered users to log in by entering their registered email and password.

Map Explorer

[R3] The platform shall display an interactive map centered on the user's current location.

[R4] The platform shall allow users to input an "Origin" and "Destination" of the desired path.

[R5] The platform shall allow users to swap the origin and destination inputs.

[R6] The platform shall calculate and display the top 5 paths options ranked by score.

[R7] The platform shall visualize the geometry, obstacles and scores of the top 5 paths on the map.

[R8] When the origin coincides with the current location, the platform shall allow users to start the navigation mode for the currently selected path.

[R9] When the navigation is started, the platform shall visualize the user's position in real-time.

[R10] When the navigation is started, the platform shall display a dynamic navigation overlay providing the name of the street being traversed and the estimated speed.

[R11] The platform shall allow users to interrupt the active navigation mode at any time.

[R12] When the navigation is completed, the platform shall present to registered users a summary screen containing statistics and the map of the path taken.

[R13] When the navigation is completed, the platform shall allow registered users to modify the path status and the obstacles.

Active Path Recording

[R14] The platform shall allow registered users to start an active recording session.

[R15] The platform shall utilize device sensors to automatically detect obstacles.

[R16] The platform shall display real-time metrics during the trip, including duration, current speed and distance traveled.

[R17] The platform shall allow registered users to stop the recording and immediately transition to the summary screen to review the captured data.

[R18] The platform shall allow registered users to modify status and obstacles in the captured data summary.

[R19] The platform shall allow registered users to publish the captured path along with its data.

Manual Path Creation

- [R20] The platform shall allow registered users to manually draw the path on the map.
- [R21] The platform shall allow registered users to manually place obstacle markers along the drawn path.
- [R22] The platform shall allow registered users to set the status of the drawn path.
- [R23] The platform shall allow registered users to publish the drawn path along with its data.

Trips & Published Paths

- [R24] The platform shall display a dashboard listing the user's historical trips and their data, sorted chronologically.
- [R25] The platform shall allow registered users to view a list of paths published by them.
- [R26] The platform shall allow registered users to delete any of their published paths.

User Profile

- [R27] The platform shall display a user profile screen summarizing the main aggregate statistics.

4.2.1 Use Cases Diagrams

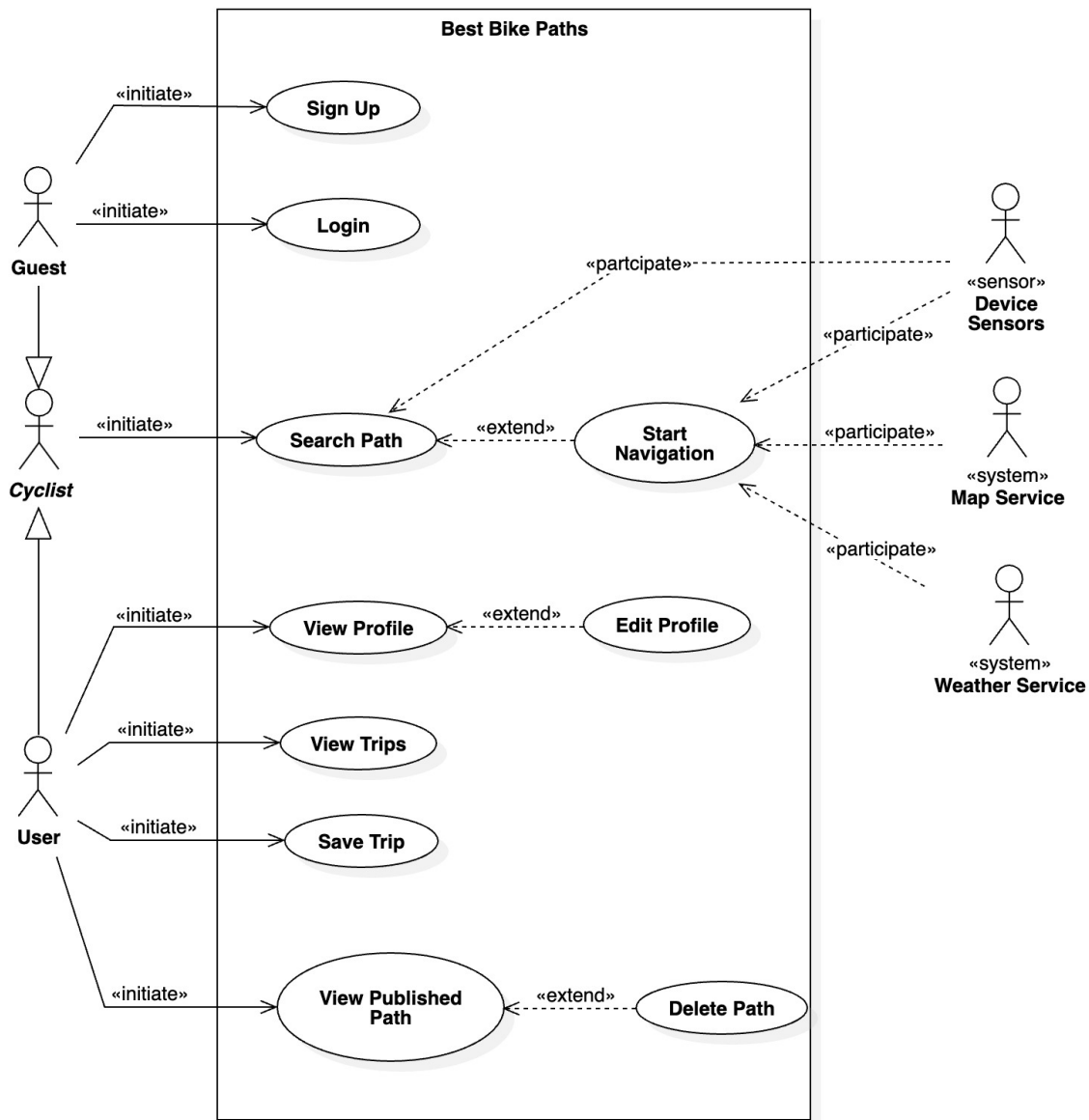


Figure 8: General Utilities Use Case Diagram

This Use Case Diagram depicts the system's functionalities organized through a generalization hierarchy. The abstract actor **Cyclist** represents the baseline capability of searching for paths, which is common to all users. Two actors inherit from this base:

- The **Guest** (unauthenticated), who extends the base capability with access-control operations.
- The **User** (authenticated), who extends the base capability with the full suite of platform features.

Finally, the diagram illustrates the need for device sensors and the participation of external services (Map and Weather Services) in supporting these functionalities.

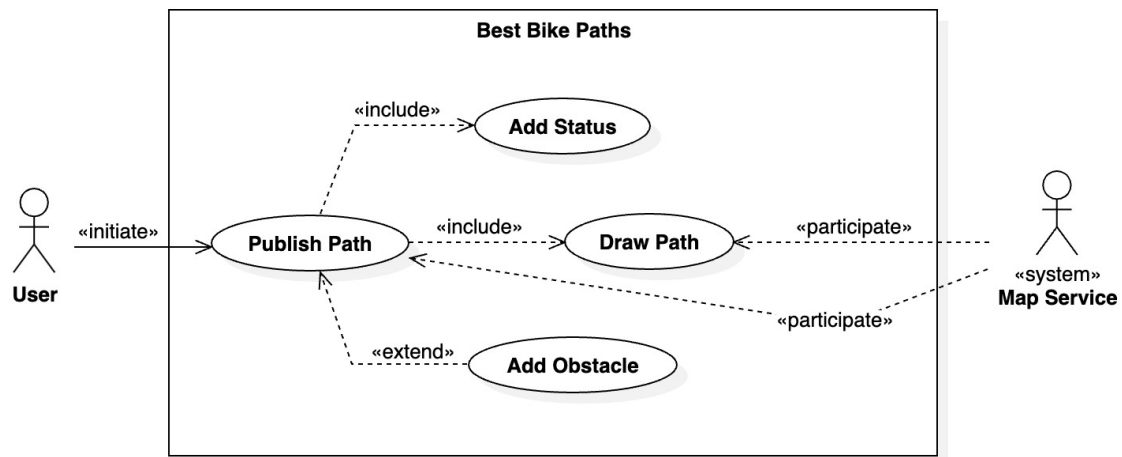


Figure 9: Manual Mode Use Case Diagram

This diagram illustrates the manual path creation process centered on the 'Publish Path' use case. This main activity mandatorily includes both 'Draw Path' (to define the path geometry) and 'Add Status' (to assign a status). Additionally, the process can be optionally extended by 'Add Obstacle' to allow the user to report specific issues on the map.

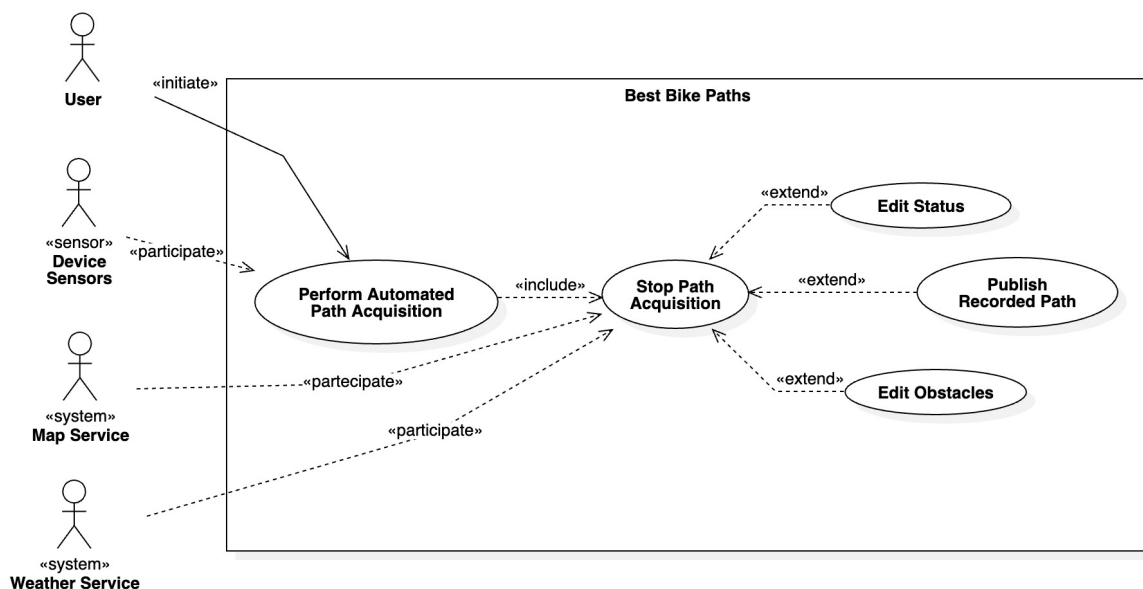


Figure 10: Automated Mode Use Case Diagram

This diagram illustrates the automated path acquisition process. The Device Sensors actor refers to the device's GPS, accelerometer, and gyroscope, which are essential for tracking the user's movement and detecting path quality in real-time.

4.2.2 Use Cases

UC-01	Sign Up
Actors	Guest
Entry Condition	The Guest is not currently logged in.
Event Flow	1. The Guest opens the login screen. 2. The Guest clicks "Sign up". 3. The Guest enters the required information. 4. The Guest presses the "Sign up" button. 5. The platform validates the entered information. 6. The platform sends a confirmation email to the provided email address. 7. The Guest confirms the registration by clicking on the link in the email. 8. The platform confirms the registration and activates the account. 9. The Guest is redirected to the login screen.
Exit Condition	A new account is created and the Guest is now in the login screen.
Exception	• The Guest provides incorrect information. • The Guest does not confirm the registration. • The email is already in use.

Table 1: [UC-01]: Sign Up Use Case

UC-02	Login
Actors	Guest
Entry Condition	The Guest is not currently logged in.
Event Flow	1. The Guest opens the login screen. 2. The Guest enters login credentials. 3. The Guest clicks the "Login" button. 4. The platform verifies login credentials. 5. The platform creates a login session for the guest. 6. The Guest is redirected to the platform's profile page as a logged-in User.
Exit Condition	The Guest is successfully authenticated as a User.
Exception	• The credentials provided are incorrect.

Table 2: [UC-02]: Login Use Case

UC-03	Search Path
Actors	Cyclist, Device Sensors
Entry Condition	The Cyclist is viewing the platform's homepage and the map is loaded
Event Flow	1. The Cyclist enters the destination. 2. The gps of the Device Sensors enters the current position in the origin. 3. The platform identifies the best 5 paths among those published. 4. The platform displays the best 5 paths on the map. 5. The Cyclist selects the desired path.
Exit Condition	The desired path is selected and the option to view its details is available to the Cyclist.
Exception	• No paths found.

Table 3: [UC-03]: Search Path Use Case

UC-04	Start Navigation
Actors	Cyclist, press(), Weather Service, Device Sensors
Entry Condition	The Cyclist has selected the desired path and the origin is the current location
Event Flow	1. The platform shows the "Start Navigation" button. 2. The Cyclist clicks "Start Navigation". 3. The platform displays the selected path on the map. 4. The Cyclist current position is updated by the platform using the GPS of the Device Sensors. 5. The press() provides the current street name. 6. The platform displays the cyclist current speed. 7. The platform detects, through the GPS, that the Cyclist has reached the destination. 8. The platform requests weather information related to the trip from the Weather Service 9. The platform asks the press() instructions for drawing the path on the map following the streets
Exit Condition	The platform displays the trip completed page.
Exception	• The user deviates from the active path for more than 15 seconds.

Table 4: [UC-04]: Start Navigation Use Case

UC-05	Save Trip
Actors	User
Entry Condition	The user is logged in and the navigation is completed.
Event Flow	1. The platform displays the "Trip Completed" screen with statistics, weather info, and rating options. 2. The User selects the path status. 3. The User modify the obstacles on the map. 4. The User clicks "Save Trip". 5. The platform saves the trip in the User's "Recent Trips" list. 6. The platform automatically submits the path report to the system. 7. The platform automatically update average performance metrics. 8. The User is redirected to the "Home Page" screen.
Exit Condition	The trip is saved and the "Home Page" screen is displayed to the User.
Exception	• None.

Table 5: [UC-05]: Save Trip Use Case

UC-06	View Trips
Actors	User
Entry Condition	The User is logged in.
Event Flow	1. The User opens the "Recent Trips" screen. 2. The platform displays the list of saved trips. 3. The User views the details of the trips in the list.
Exit Condition	The detailed view of the saved trips is displayed.
Exception	• None.

Table 6: [UC-06]: View Trips Use Case

UC-07	Edit Profile
Actors	User
Entry Condition	The User is logged in.
Event Flow	1. The User opens the "Profile" screen. 2. The User opens the "Settings" 3. The User clicks "Edit Profile". 4. The User modifies the information. 5. The User clicks "Confirm Changes" 6. The platform validates the new data. 7. The platform updates the profile and displays the success message.
Exit Condition	The User profile is updated with the new information.
Exception	• The User leaves a mandatory field empty and clicks "Confirm Changes"

Table 7: [UC-07]: Edit Profile Use Case

UC-08	Delete Path
Actors	User
Entry Condition	The User is logged in.
Event Flow	1. The User opens the "Published path" screen. 2. The User selects the path to delete. 3. The platform displays the path details. 4. The User clicks the "Delete" button. 5. The platform asks for confirmation. 6. The User confirms the deletion. 7. The platform removes the path and updates the list.
Exit Condition	The path is permanently deleted from the published paths list.
Exception	• None.

Table 8: [UC-08] Delete Path Use Case

UC-09	Publish Path
Actors	User, Map Service
Entry Condition	The User is logged in and is viewing the homepage.
Event Flow	1. The User clicks the "+" button to create a new path. 2. The platform loads the "Draw Mode" interface. 3. The User draw the path. 4. The User clicks the "Next Step" button. 5. The platform asks to Map Service to get the name of streets. 6. The platform displays the "Review & Publish" screen. 7. The User selects the status. 8. The User clicks "Publish Path". 9. The platform validates and makes the path available to the community.
Exit Condition	The path is published and appears in the User's "Published Paths" screen.
Exception	• The platform detects that an identical path already exists.

Table 9: [UC-09]: Publish Path Use Case

UC-10	Draw Path
Actors	User, Map Service
Entry Condition	The User is in the 'Draw Path' mode within the manual creation interface and the interactive map is loaded.
Event Flow	1. The User taps on the map to place the starting point. 2. The User continues tapping to trace the path segments. 3. The Map Service renders the path line in real-time. 4. The platform shows the "Total Km". 5. The User completes the tracing.
Exit Condition	The geometric path is defined on the map.
Exception	• None

Table 10: [UC-10]: Draw Path Use Path

UC-11	Add Obstacle
Actors	User
Entry Condition	The User is in the manual creation interface, the interactive map is loaded, and the path is partially or fully drawn.
Event Flow	1. The User clicks on the "Obstacles" tab. 2. The platform switches to "Obstacle Mode". 3. The User taps a specific point along the drawn path. 4. The platform places an obstacle marker on the coordinate.
Exit Condition	An obstacle is flagged on the map.
Exception	• The user adds an obstacle outside the path

Table 11: [UC-11]: Add Obstacle Use Case

UC-12	Perform Automated Path Acquisition
Actors	User, Device Sensors
Entry Condition	The User is logged in and is viewing the homepage.
Event Flow	1. The User clicks the "Lightning" button to start the acquisition. 2. The platform starts the timer. 3. The platform initializes the "Automated Mode" interface. 4. The platform begins recording distance, speed and obstacles in real-time using the Device Sensors. 5. The platform continuously updates the metrics (distance, obstacles, speed). 6. The User stops the path acquisition by clicking the "Stop button".
Exit Condition	The "Automated Mode" is stopped and the User is viewing the "Trip Summary".
Exception	• None

Table 12: [UC-12]: Perform Automated Path Acquisition

UC-13	Stop Path Acquisition
Actors	User, Map Service, Weather Service
Entry Condition	The User is logged in and the "Automated Mode" is active.
Event Flow	1. The User stops the path acquisition by clicking the "Stop button". 2. The platform stops the timer. 3. The platform requests weather information related to the trip from the Weather Service. 4. The platform asks the Map Service instructions for drawing the path on the map following the streets. 5. The platform asks to Map Service to get the name of the streets. 6. The platform displays the "Trip Summary" interface.
Exit Condition	The User is viewing the "Trip Summary"
Exception	• None

Table 13: [UC-13]: Stop Path Acquisition Use Case

UC-14	Publish Recorded Path
Actors	User
Entry Condition	The User is viewing the "Trip Summary" interface with the recorded path data.
Event Flow	1. The User clicks the "Publish Path" button. 2. The platform stores the path data (geometry, status, obstacles) in the system. 3. The platform validates and makes the path available to the community.
Exit Condition	The path is published and appears in the User's "Published Paths" screen.
Exception	• The platform detects that an identical path already exists.

Table 14: [UC-14]: Publish Recorded Path Use Case

4.2.3 Sequence Diagrams

The functions used in the sequence diagrams follow these conventional meanings:

- **press(Button)**: Represents the action of a user pressing Button.
- **submit(Content)**: Indicates a user uploading Content to the platform.
- **showPage(Page)**: Displays the content of a specific Page.
- **showError(Error)**: Displays an error message indicating an issue that occurred.
- **check()**: Verifies the validity of the information provided by the user.
- **send(ConfirmationEmail)**: The system sends a confirmation email to user.
- **confirmEmail()**: The user confirms the email, which triggers a confirmation signal to the system.
- **register(User)**: The system registers the user.
- **update(Data)**: Refers to internal processes that update database records with Data.
- **enter()**: Represents the action of a user entering an input.
- **getCurrentPosition()**: The system retrieves user's current position through Device Sensors.
- **currentPosition()**: The Device Sensors send user's current position to the system.
- **findBestPaths(Origin, Destination)**: Identifies and returns the 5 paths with the best performance scores connecting the selected locations.
- **selectPath(Path)**: Represents the user choosing one of the displayed paths to view its details or start navigation.
- **getWeatherInfo()**: Requests meteorological data (temperature, conditions) for the specific trip.
- **weatherInfo()**: Retrieves the requested weather information from the external service. It returns the weather metrics if found, or a null/empty value if the data is unavailable, ensuring the process continues without handling an exception.
- **insert(Object)**: Represents the user placing a marker on the map.
- **drawMarkerOnMap()**: The system draws a marker on the coordinate where the user clicked.
- **displayCursorOnMap()**: The system displays a cursor in the user's current position.
- **displayCurrentStreet()**: The system displays the current street name during navigation.
- **checkCyclistPosition(path, currentPosition)**: The system check that the user is following the path selected for the navigation.
- **calculateMetrics()**: The system calculate the metrics during the navigation.
- **mostFrequentWeather()**: The system computes the most frequent weather in the trip area.
- **delete()**: The system deletes a published path.
- **drawPath()**: The system draws a path on the map.
- **drawPathOnMap()**: The system draws a path on map.

- **getPathGeometry()**: The system retrieves path coordinates.
- **pathCoordinates()**: The map service returns path coordinates.
- **showStatusUpdated()**: The system shows the updated status.
- **modify(obstacle)**: The user modify an obstacle.
- **save()**: The system saves data on the database.
- **obstaclesAdded()**: The system add the obstacles.
- **getSensorsData()**: The system retrieves sensors data.
- **sensorsData()**: The device sensors return sensor data.
- **getStreetNames()**: The system retrieves the street names of a path.
- **streetNames()**: The map service return the street names of a path.
- **DisplayInfoAndTimer()**: The system displays info and timer of the trip.
- **checkObstacle(acc,giros)**: The system check the obstacles on the path during the automated mode.
- **fetchData()**: A generic system request to retrieve specific data from the database.
- **showMessage()**: Shows a general message on the screen.
- **startTimer()**: Internal process that starts the timer.
- **stopTimer()**: Internal process that stops the timer.
- **pathDeviated()**: Device detects that user is deviating from active path.

[SD1] Sign Up Sequence Diagram

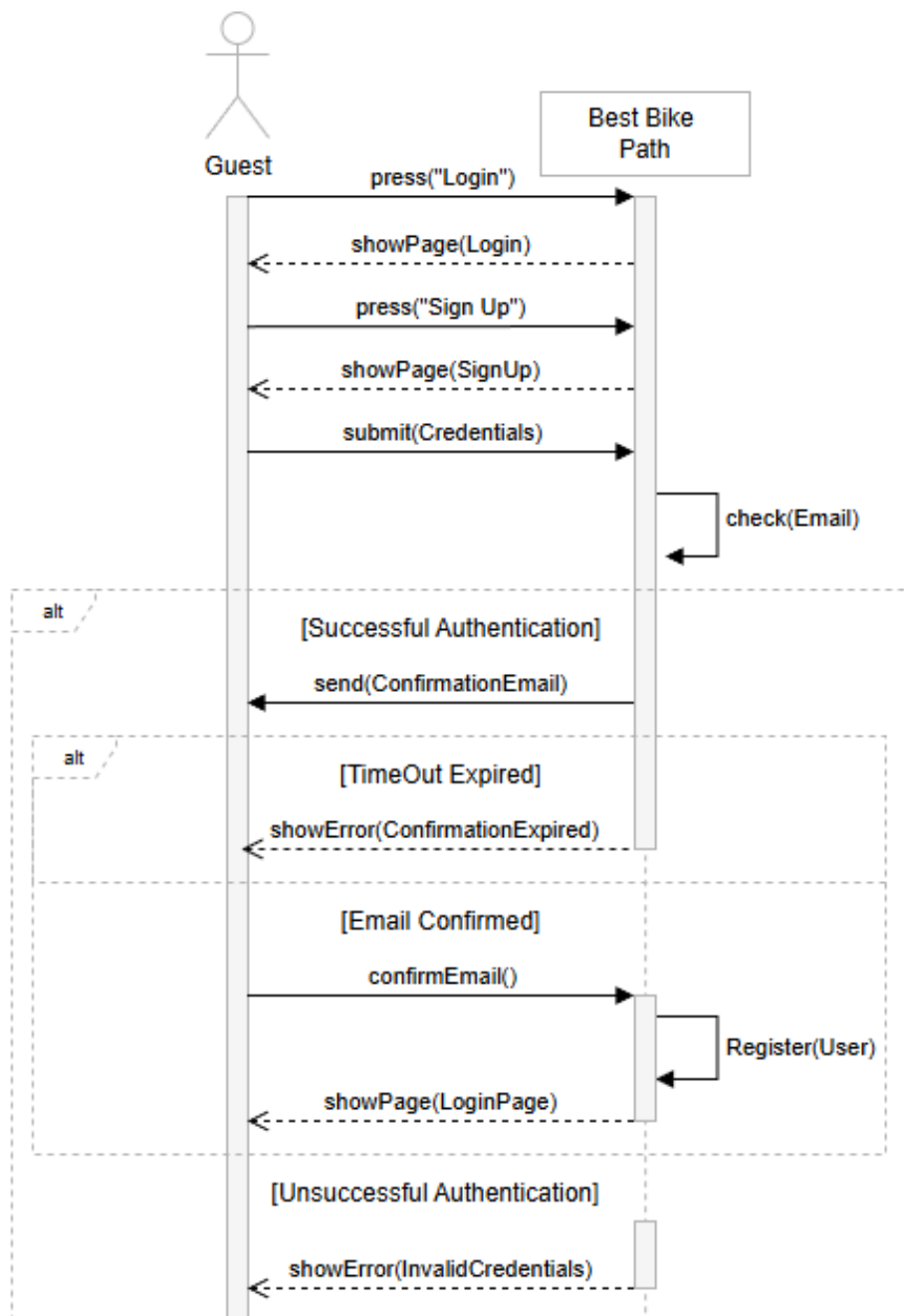


Figure 11: [SD1]: Sign Up Sequence Diagram

[SD2] Login Sequence Diagram

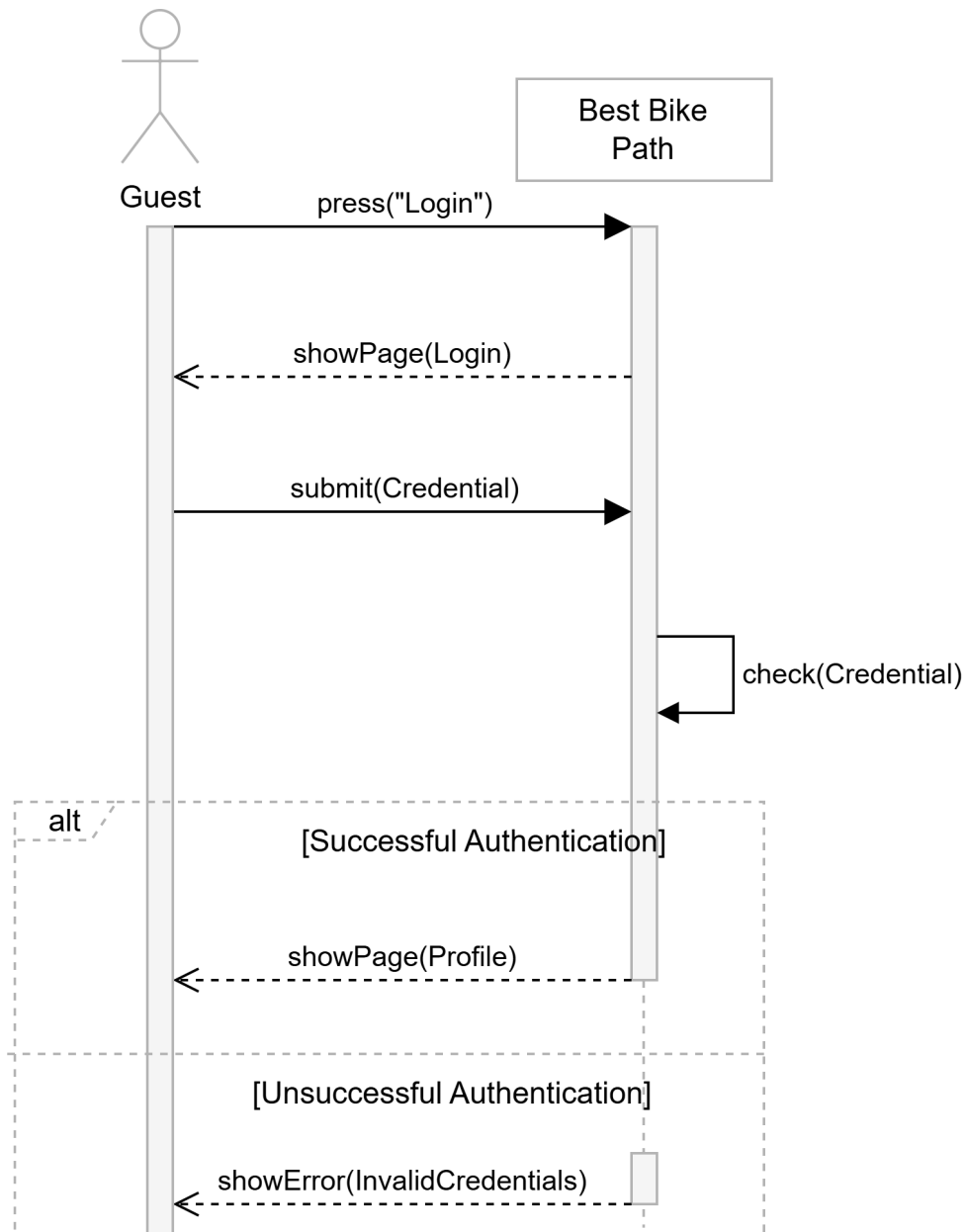


Figure 12: [SD2]: Login Sequence Diagram

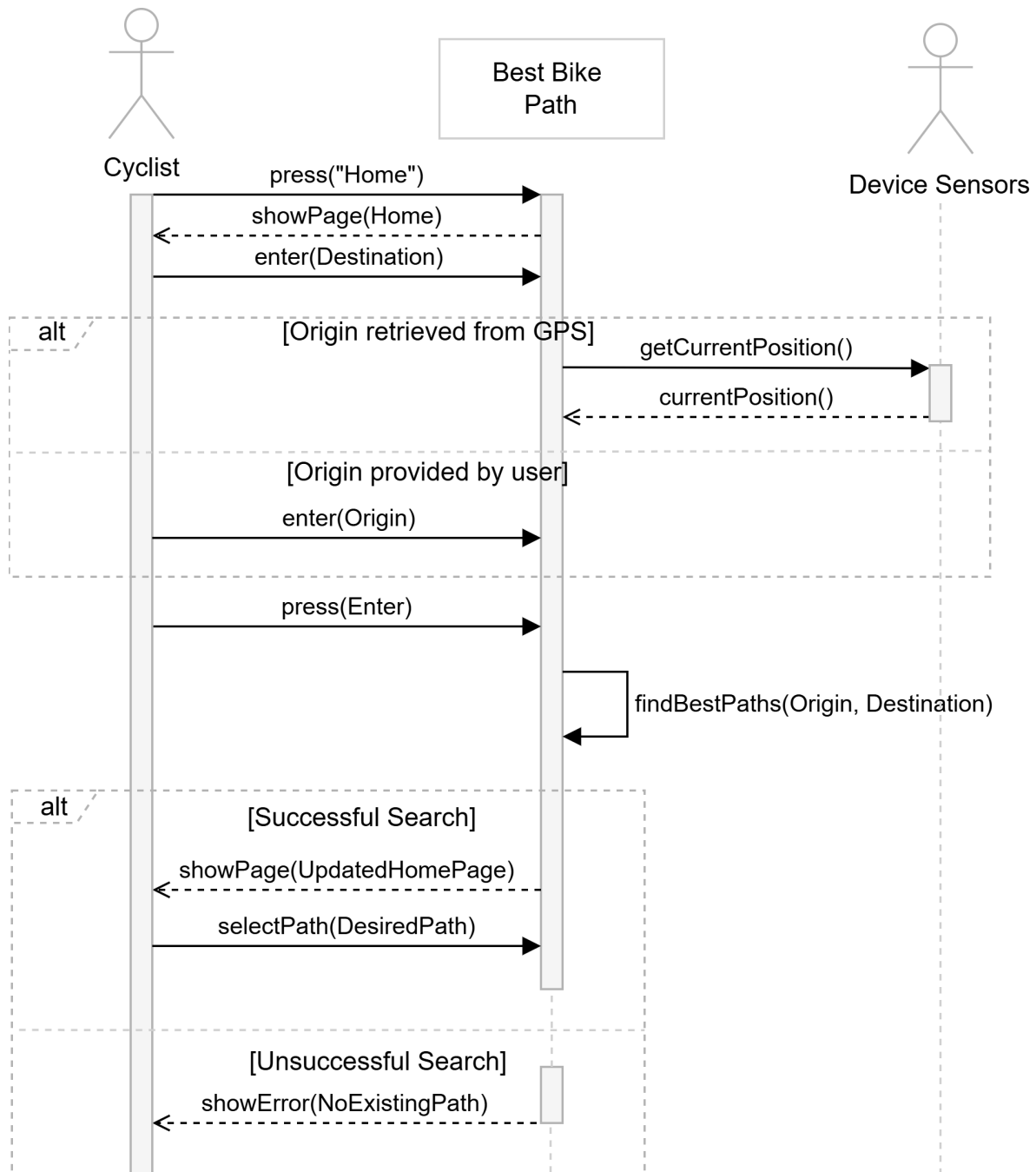
[SD3] Search Path Sequence Diagram

Figure 13: [SD3]: Search Path Sequence Diagram

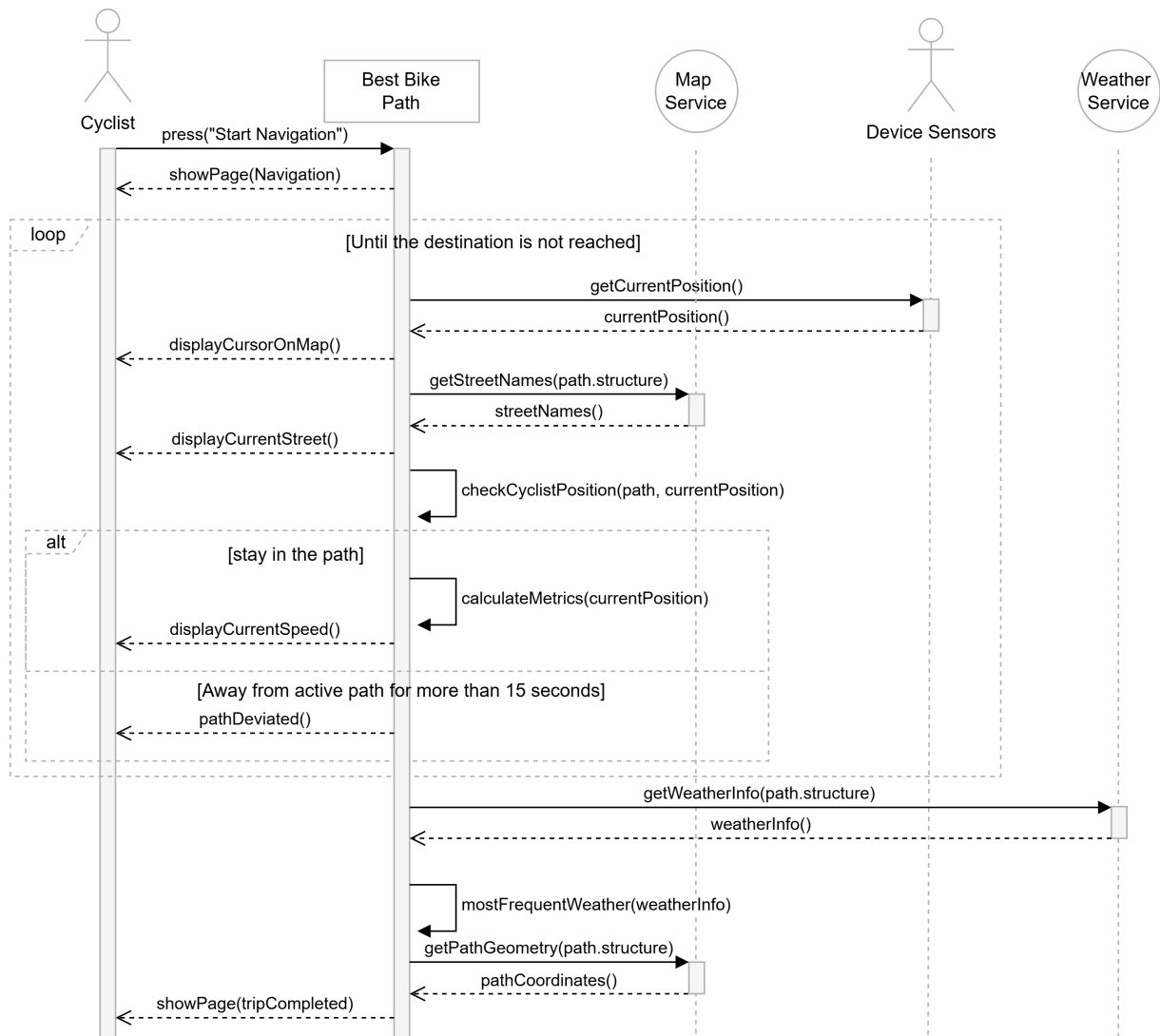
[SD4] Start Navigation Sequence Diagram

Figure 14: [SD4]: Start Navigation Sequence Diagram

[SD5] Save Trip Sequence Diagram

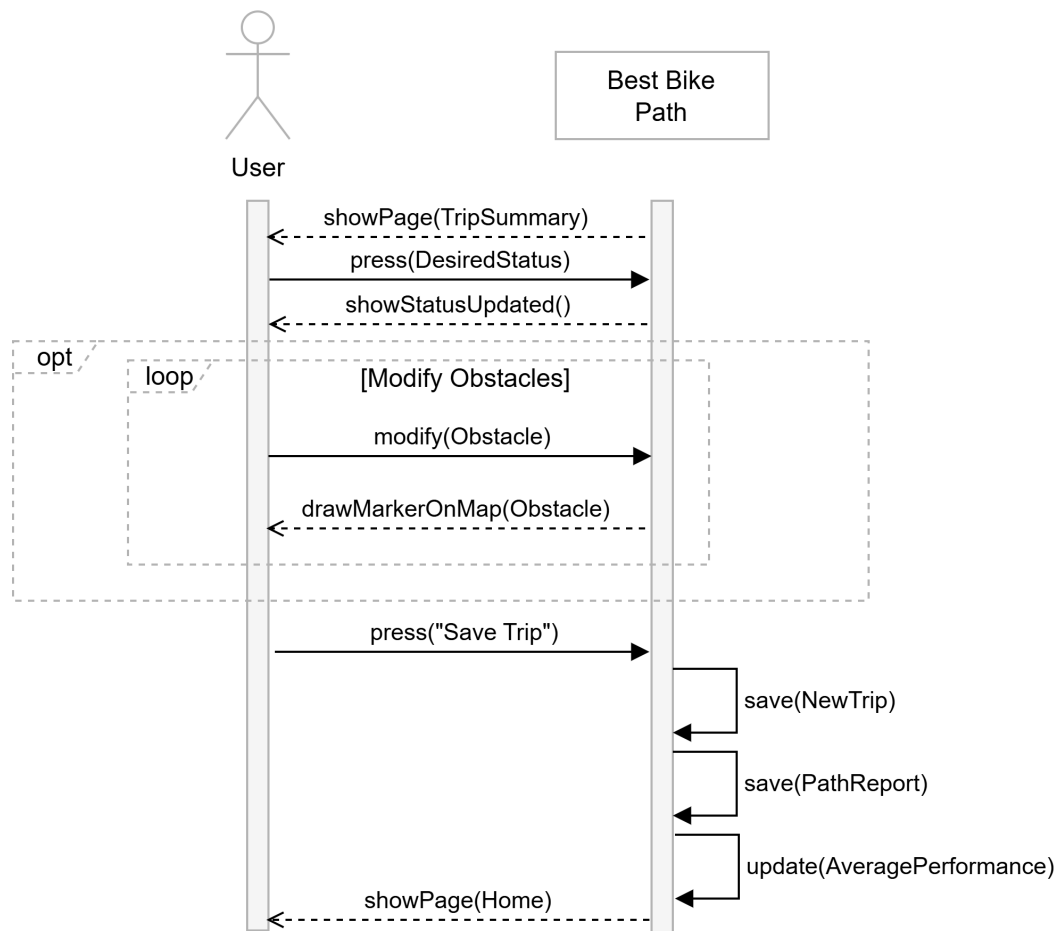


Figure 15: [SD5]: Save Trip Sequence Diagram

[SD6] View Trip Sequence Diagram

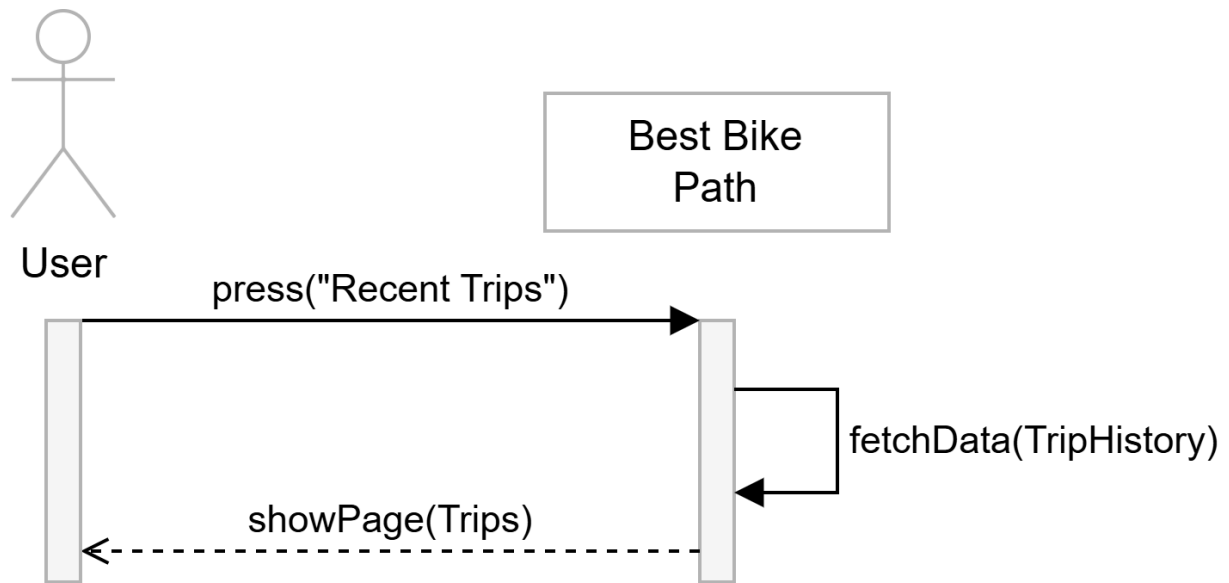


Figure 16: [SD6]: View Trip Sequence Diagram

[SD7] Edit Profile Sequence Diagram

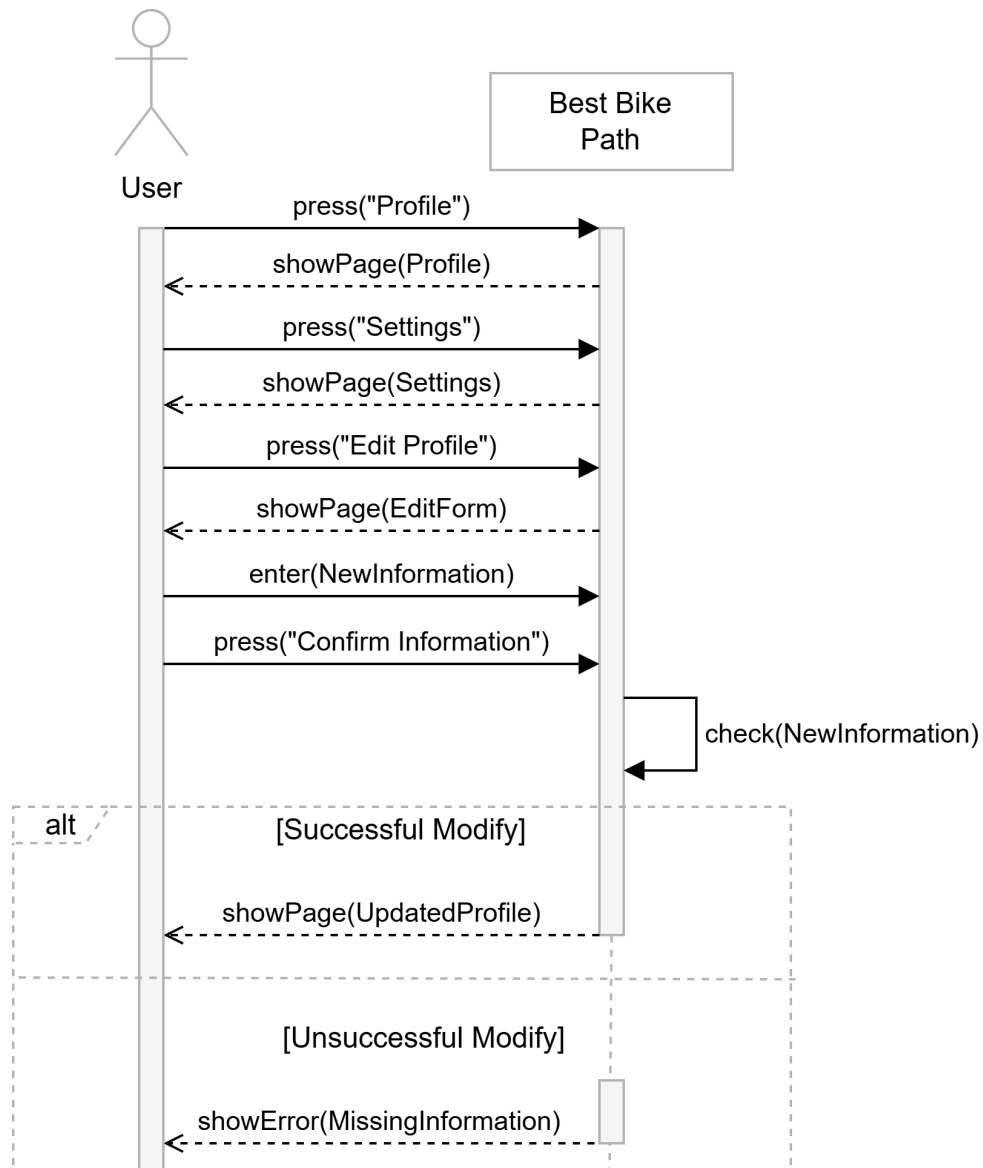


Figure 17: [SD7]: Edit Profile Sequence Diagram

[SD8] Delete Path Sequence Diagram

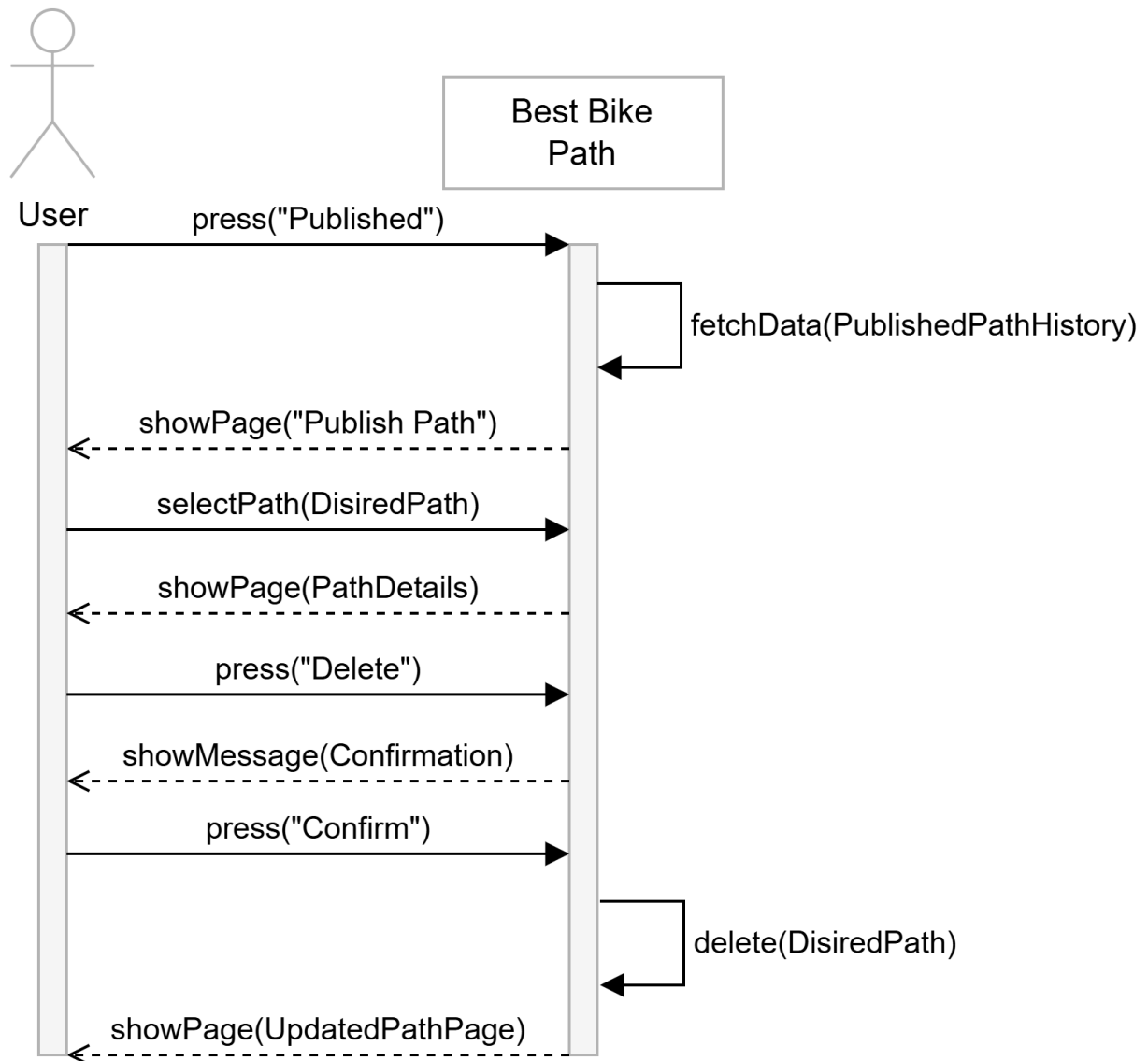


Figure 18: [SD8]: Delete Path Sequence Diagram

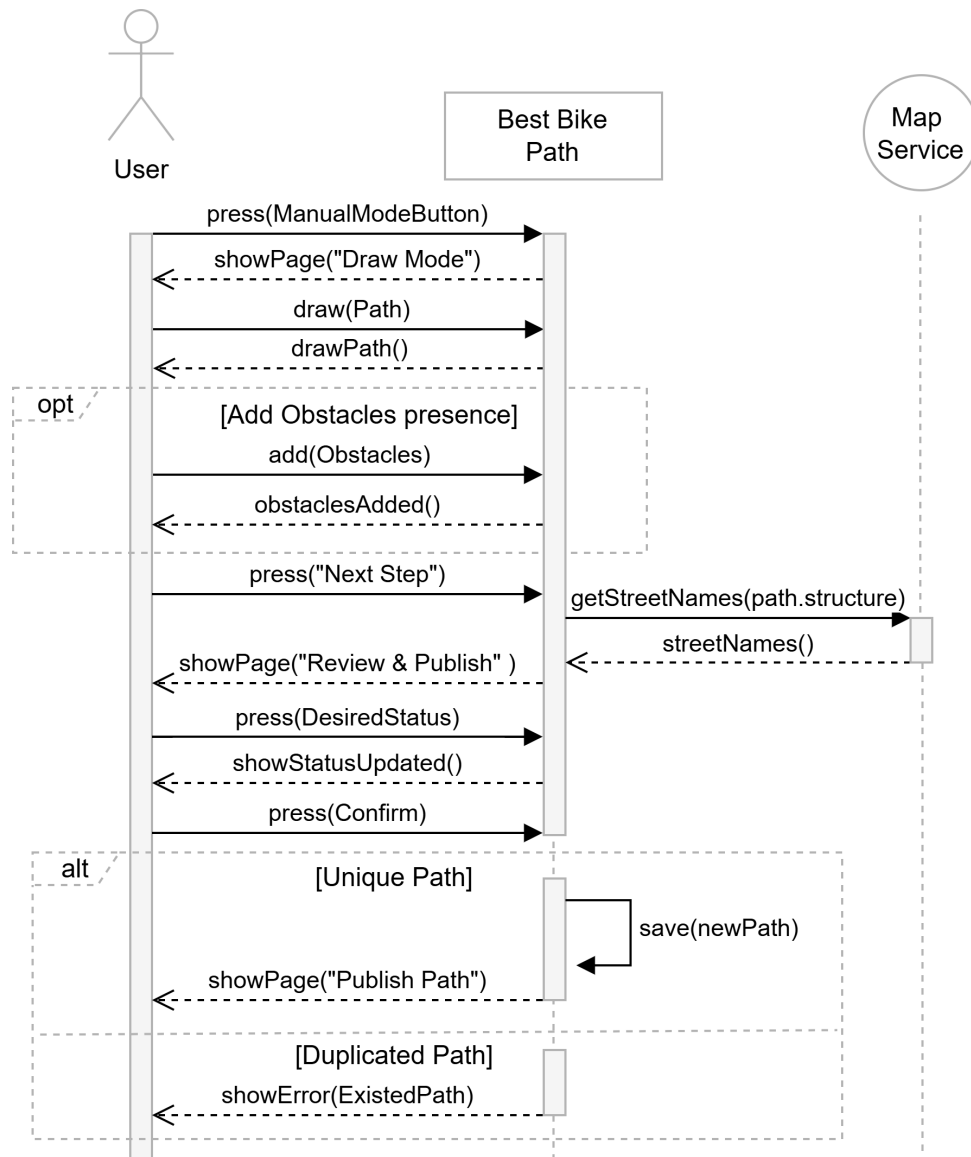
[SD9] Publish Path Sequence Diagram

Figure 19: [SD9]: Publish Path Sequence Diagram

Notes:

- The *draw(PathPoint)* function is reported with details in **SD10**.
- The *add(Obstacle)* function is reported with details in **SD11**.

[SD10] Draw Path Sequence Diagram

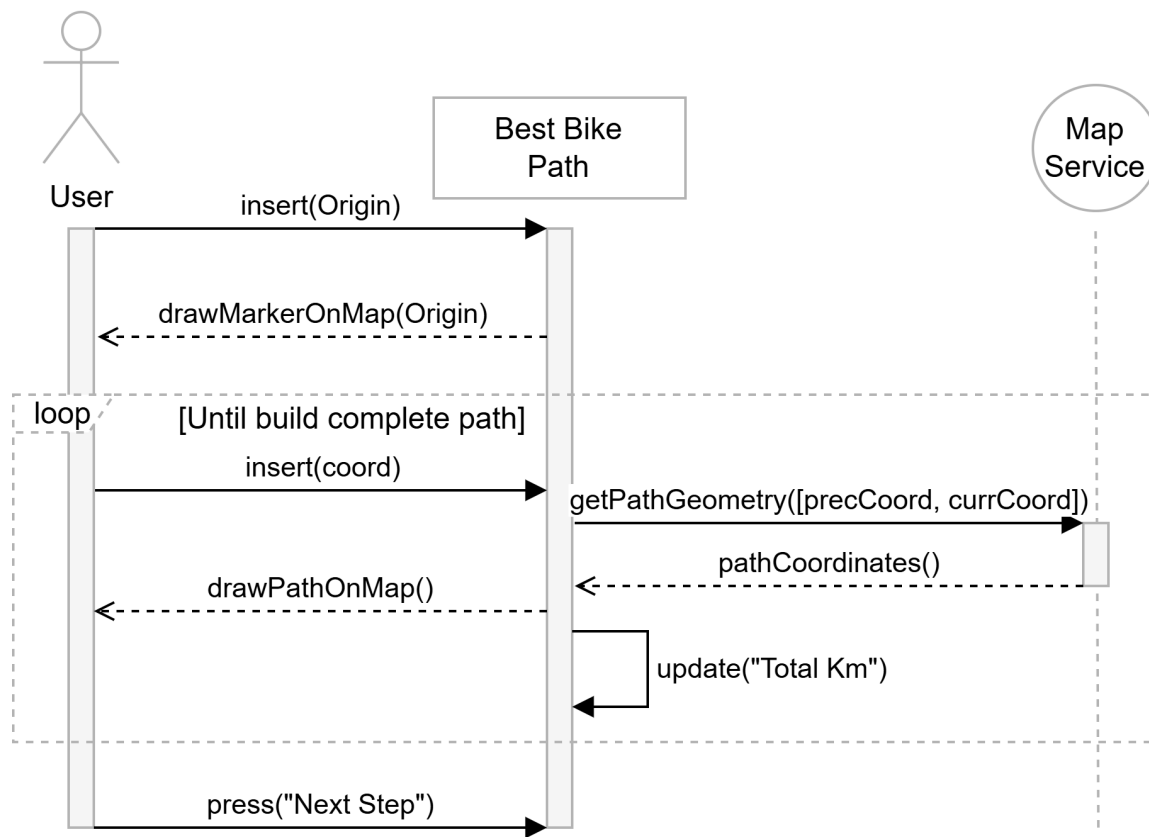


Figure 20: [SD10]: Draw Path Sequence Diagram

[SD11] Add Obstacle Sequence Diagram

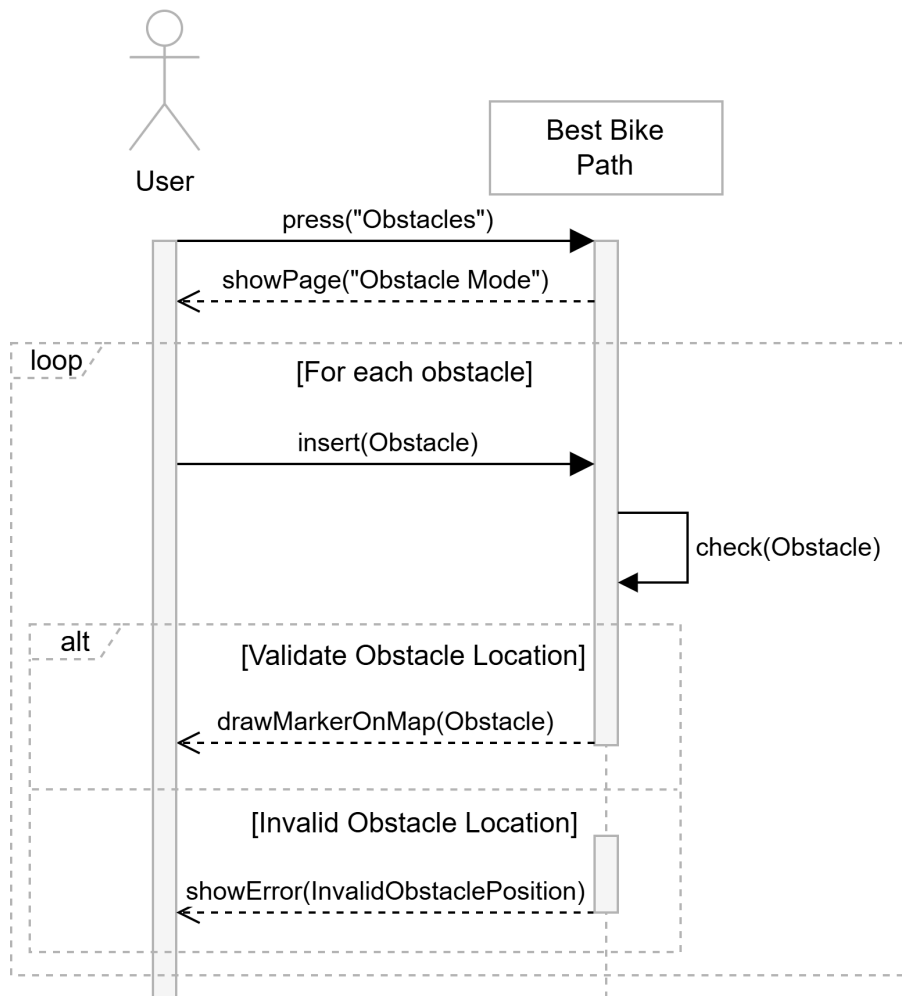


Figure 21: [SD11]: Add Obstacle Sequence Diagram

[SD12] Perform Automated Path Acquisition Sequence Diagram

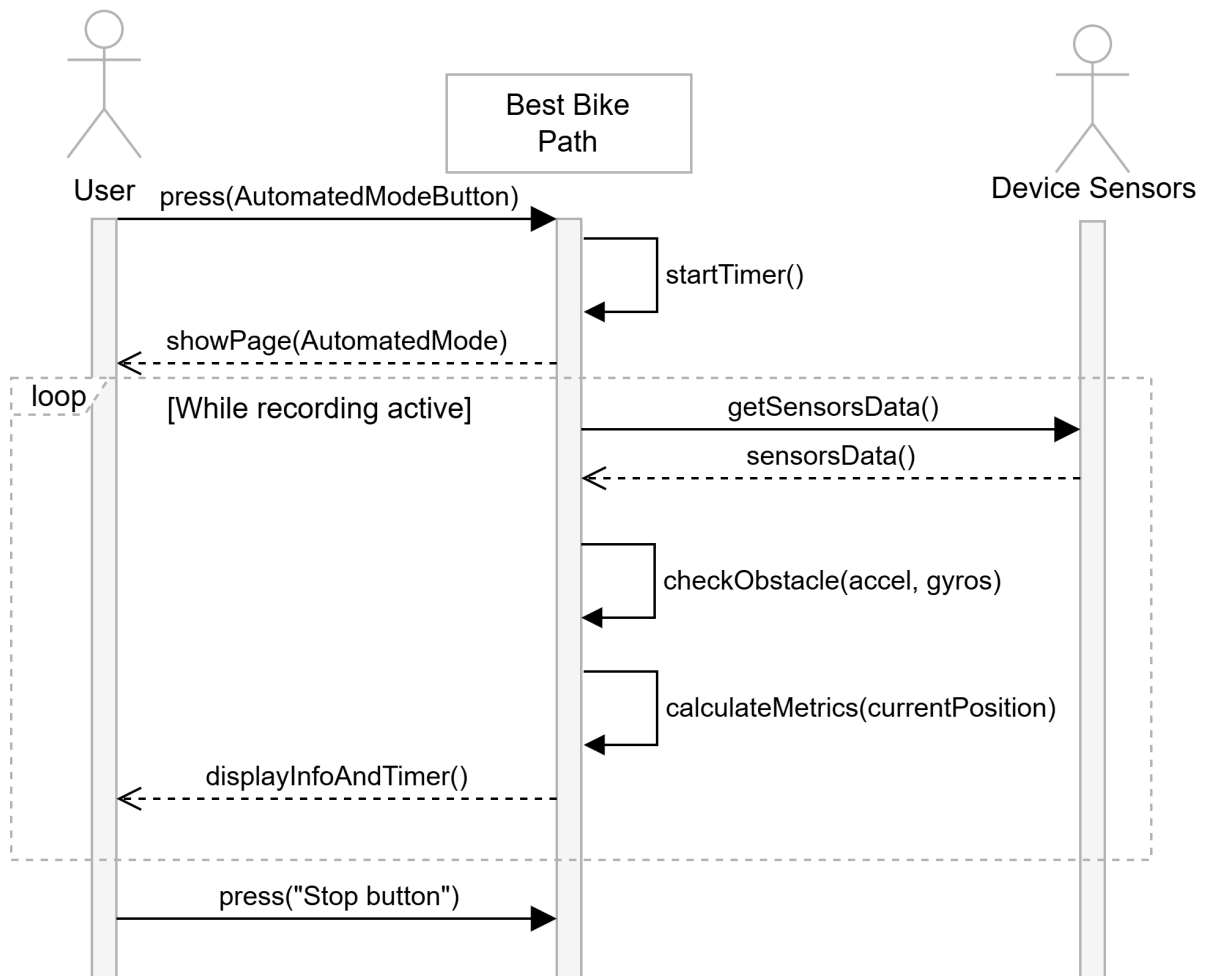


Figure 22: [SD12]: Perform Automated Path Acquisition Sequence Diagram

[SD13] Stop Path Acquisition Sequence Diagram

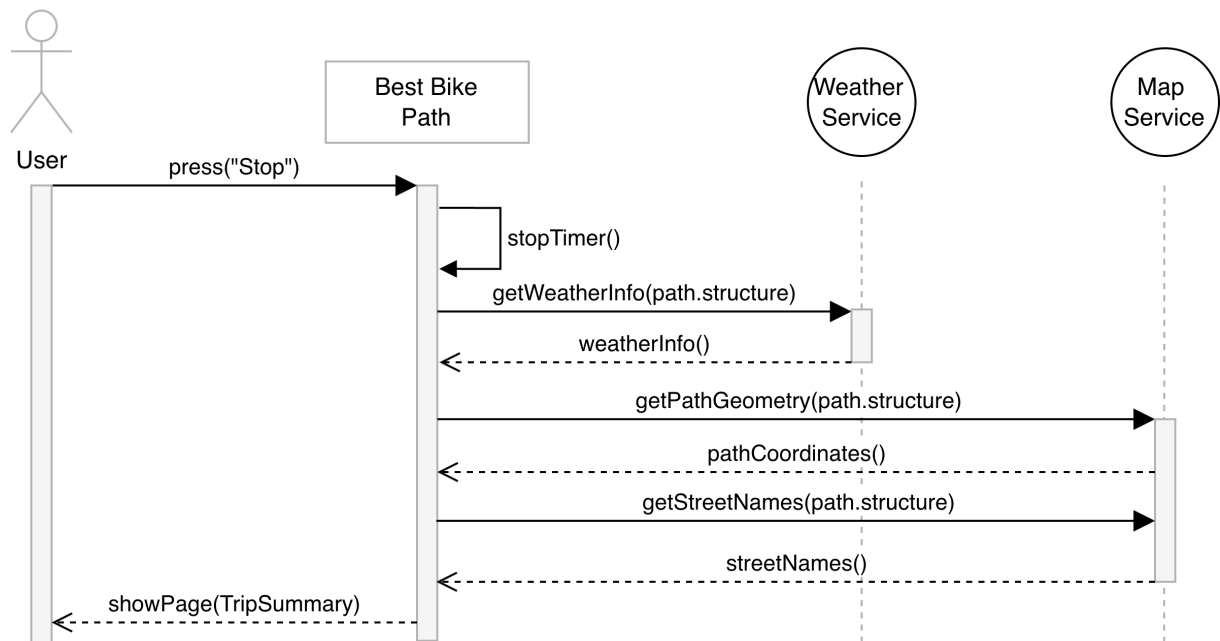


Figure 23: [SD13]: Stop Path Acquisition Sequence Diagram

[SD14] Publish Recorded Path Sequence Diagram

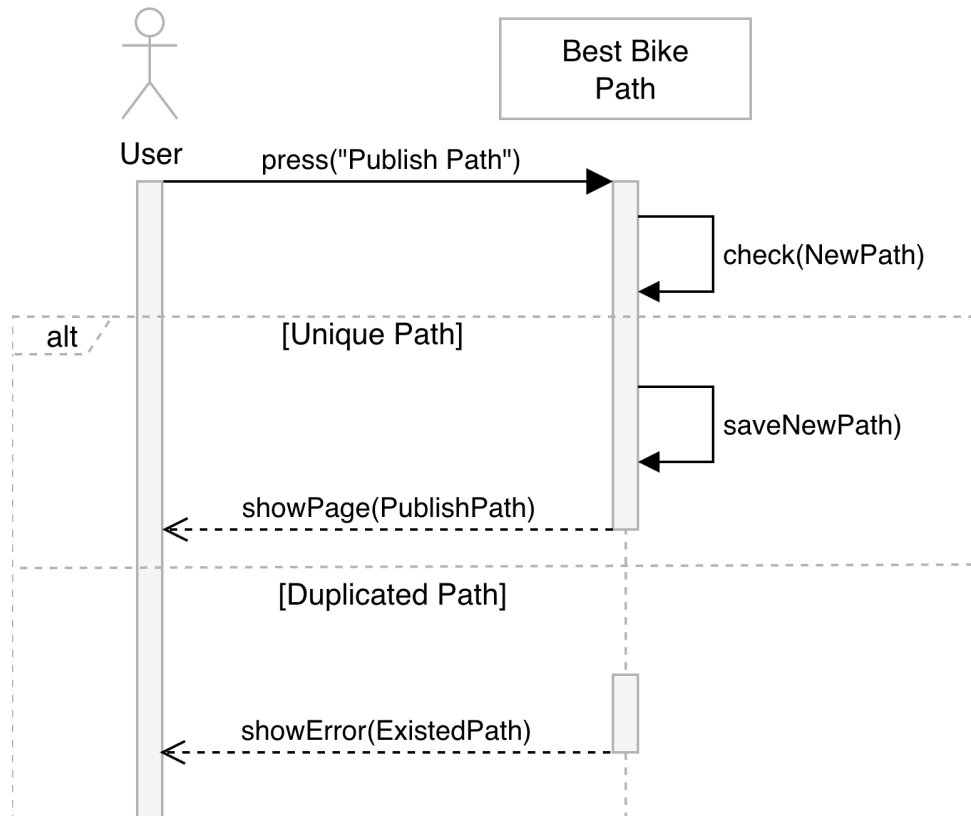


Figure 24: [SD14]: Publish Recorded Path Sequence Diagram

4.2.4 Requirements Mapping

G1: Registered users would like to maintain a history of their trips.	
[D1]	The user's device is equipped with a GPS, an accelerometer and a gyroscope.
[D2]	The user has a working internet connection.
[R1]	The platform shall allow new users to register an account by providing personal information.
[R2]	The platform shall allow registered users to log in by entering their registered email and password.
[R8]	When the origin coincides with the current location, the platform shall allow users to start the navigation mode for the currently selected path.
[R9]	When the navigation is started, the platform shall visualize the user's position in real-time.
[R10]	When the navigation is started, the platform shall display a dynamic navigation overlay providing the name of the street being traversed and the estimated speed.
[R12]	When the navigation is completed, the platform shall present to registered users a summary screen containing statistics and the map of the path taken.
[R14]	The platform shall allow registered users to start an active recording session.
[R16]	The platform shall display real-time metrics during the trip, including duration, current speed and distance traveled.
[R17]	The platform shall allow registered users to stop the recording and immediately transition to the summary screen to review the captured data.
[R24]	The platform shall display a dashboard listing the user's historical trips and their data, sorted chronologically.

Table 15: Goal 1 mapping

G2: Registered users should have access to performance metrics about their trips.	
[D1]	The user's device is equipped with a GPS, an accelerometer and a gyroscope.
[D2]	The user has a working internet connection.
[D5]	The external weather information service is reliable.
[D6]	The external press() is available and provides accurate topographical data.
[R1]	The platform shall allow new users to register an account by providing personal information.
[R2]	The platform shall allow registered users to log in by entering their registered email and password.
[R9]	When the navigation is started, the platform shall visualize the user's position in real-time.
[R10]	When the navigation is started, the platform shall display a dynamic navigation overlay providing the name of the street being traversed and the estimated speed.
[R12]	When the navigation is completed, the platform shall present to registered users a summary screen containing statistics and the map of the path taken.
[R16]	The platform shall display real-time metrics during the trip, including duration, current speed and distance traveled.
[R17]	The platform shall allow registered users to stop the recording and immediately transition to the summary screen to review the captured data.
[R24]	The platform shall display a dashboard listing the user's historical trips and their data, sorted chronologically.
[R27]	The platform shall display a user profile screen summarizing the main aggregate statistics.

Table 16: Goal 2 mapping

G3: When available, registered users would like to be informed about meteorological information related to their trips.	
[D1]	The user's device is equipped with a GPS, an accelerometer and a gyroscope.
[D2]	The user has a working internet connection.
[D5]	The external weather information service is reliable.
[R1]	The platform shall allow new users to register an account by providing personal information.
[R2]	The platform shall allow registered users to log in by entering their registered email and password.
[R16]	The platform shall display real-time metrics during the trip, including duration, current speed and distance traveled.
[R24]	The platform shall display a dashboard listing the user's historical trips and their data, sorted chronologically.

Table 17: Goal 3 mapping

G4: Registered users would like to share information about specific bike paths.	
[D2]	The user has a working internet connection.
[D3]	The user provides truthful information regarding the path to be published.
[D4]	The user provides truthful reports regarding paths.
[D6]	The external press() is available and provides accurate topographical data.
[R1]	The platform shall allow new users to register an account by providing personal information.
[R2]	The platform shall allow registered users to log in by entering their registered email and password.
[R18]	The platform shall allow registered users to modify status and obstacles in the captured data summary.
[R19]	The platform shall allow registered users to publish the captured path along with its data.
[R20]	The platform shall allow registered users to manually draw the path on the map.
[R21]	The platform shall allow registered users to manually place obstacle markers along the drawn path.
[R22]	The platform shall allow registered users to set the status of the drawn path.
[R23]	The platform shall allow registered users to publish the drawn path along with its data.
[R25]	The platform shall allow registered users to view a list of paths published by them.
[R26]	The platform shall allow registered users to delete any of their published paths.

Table 18: Goal 4 mapping

G5: Users would like to find the optimal bike path between an origin and a destination by comparing viable alternatives based on efficiency and status.	
[D1]	The user's device is equipped with a GPS, an accelerometer and a gyroscope.
[D2]	The user has a working internet connection.
[D6]	The external press() is available and provides accurate topographical data.
[R3]	The platform shall display an interactive map centered on the user's current location.
[R4]	The platform shall allow users to input an "Origin" and "Destination" of the desired path.
[R5]	The platform shall allow users to swap the origin and destination inputs.
[R6]	The platform shall calculate and display the top 5 paths options ranked by score.
[R7]	The platform shall visualize the geometry, obstacles and scores of the top 5 paths on the map.

Table 19: Goal 5 mapping

G6: Users would like to access reliable and up-to-date shared information about bike paths.	
[D1]	The user's device is equipped with a GPS, an accelerometer and a gyroscope.
[D2]	The user has a working internet connection.
[D3]	The user provides truthful information regarding the path to be published.
[D4]	The user provides truthful reports regarding paths.
[D6]	The external press() is available and provides accurate topographical data.
[R6]	The platform shall calculate and display the top 5 paths options ranked by score.
[R7]	The platform shall visualize the geometry, obstacles and scores of the top 5 paths on the map.
[R13]	When the navigation is completed, the platform shall allow registered users to modify the path status and the obstacles.
[R15]	The platform shall utilize device sensors to automatically detect obstacles.
[R26]	The platform shall allow registered users to delete any of their published paths.

Table 20: Goal 6 mapping

	G1	G2	G3	G4	G5	G6
D1	x	x	x		x	x
D2	x	x	x	x	x	x
D3				x		x
D4				x		x
D5		x	x			
D6		x		x	x	x
R1	x	x	x	x		
R2	x	x	x	x		
R3					x	
R4					x	
R5					x	
R6					x	x
R7					x	x
R8	x					
R9	x	x				
R10	x	x				
R11						
R12	x	x				
R13						x
R14	x					
R15						x
R16	x	x	x			
R17	x	x				
R18				x		
R19				x		
R20				x		
R21				x		
R22				x		
R23				x		
R24	x	x	x			
R25				x		
R26				x		x
R27		x				

Table 21: Goals Mapping Table

4.3 Performance Requirements

Given the application's focus on real-time navigation and outdoor activities, efficient resource usage and low latency are essential to ensure safety and usability. To ensure an optimal User experience:

- The system shall update real-time dashboard metrics (Speed, Distance, Obstacles) within 1 second of receiving data from the device sensors;
- The system shall respond to general User interface requests within 2 seconds under normal load conditions;
- The system shall support at least 10,000 concurrent Users navigating or recording simultaneously;
- Database queries shall be completed within 1.5 seconds;
- The system shall be capable of processing and saving trip summaries for at least 100 Users per second during peak hours;
- The Path Calculation process (Top 5 options) shall be completed and displayed to the User within 3 seconds under normal load conditions;
- The application shall not consume more than 15% of the battery capacity per hour on a standard reference device while in "Active Navigation" mode (screen on and GPS tracking enabled), to prevent battery drainage during long rides.

4.4 Design Constraints

This section explain the different constraints that the platform must respect such as the standard compliance for the data protection and hardware limitations.

4.4.1 Standards Compliance

The Platform operates at the intersection of sensitive user data processing and third-party proprietary services. Specifically, the system must handle Personally Identifiable Information (PII) and continuous geolocation history, which constitutes a significant privacy risk as it can reveal user habits and private locations. Consequently, the architecture must be designed to strictly comply with the General Data Protection Regulation (GDPR) and potentially the California Consumer Privacy Act (CCPA), embedding privacy-by-design principles directly into the software core. Simultaneously, the dependency on external map providers (e.g., OpenStreetMap or Google Maps) mandates rigorous adherence to their Terms of Service, requiring the software to dynamically display proper copyright attributions and strictly enforce technical limitations on offline data caching. The system must therefore be architected with the flexibility to adapt to this dual set of regulatory and licensing constraints, ensuring compliance without compromising functionality.

4.4.2 Hardware Limitations

The mobile application client relies strictly on the availability and precision of consumer-grade hardware sensors to execute its core logic. Specifically, the host device must be equipped with a GPS, an accelerometer, and a gyroscope, as the automatic detection of path obstacles and accurate real-time navigation are algorithmically dependent on the continuous data stream provided by this specific sensor suite. Furthermore, considering the prolonged nature of cycling activities, the application architecture must operate within the strict energy and thermal constraints of standard smartphones, ensuring that continuous background tracking does not deplete the battery before the trip concludes or trigger thermal throttling that would degrade system performance.

4.5 Software System Attributes

This section explains the design constraints that the platform must respect, covering both regulatory compliance for data protection and specific hardware limitations.

4.5.1 Reliability & Availability

The system is designed to ensure high availability for Users, targeting an uptime of 99.9%. To achieve this, the platform relies on redundant cloud infrastructure to minimize the risk of service interruptions. Furthermore, the mobile application is designed to be resilient to network failures; it utilizes local caching mechanisms to ensure that critical features, such as path recording, remain functional and no data is lost even when the device temporarily loses connectivity to the server.

4.5.2 Security

The system handles highly sensitive information, including real-time geolocation history and personal user profiles. Consequently, these data must be protected with strong encryption algorithms (at rest), and data in transit must always be encrypted via HTTPS to ensure security against interception. The system may be particularly susceptible to API abuse and unauthorized access attempts; hence, it should incorporate robust authentication measures (such as token-based access) and rate-limiting strategies to prevent or reduce the impact of such attacks.

4.5.3 Maintainability

The backend components should be designed to be easily scalable, maintainable, reusable, and testable to handle a growing number of cyclists and tracks. To achieve this, the system should implement well-defined architecture that allows for easy updates and modifications both on the front and the back end. The code should be readable and fully documented. Documentation should include detailed and well-analyzed API definitions, a comprehensive description of the system architecture, and should clearly define the automated testing strategies employed to prevent regressions.

4.5.4 Portability

As a mobile-centric platform, the client application must be compatible with the major mobile operating systems, specifically Android and iOS. It should efficiently utilize native hardware features (such as GPS and accelerometers) while ensuring a consistent user experience. Moreover, the interface should be optimized for use on various device form factors, ensuring flexibility and usability across both standard smartphones and larger tablets used for path planning.

5 Formal Analysis Using Alloy

The objective of this section is to validate the critical aspects of the Best Bike Paths (BBP) system using Alloy, a lightweight formal method based on relational logic. In particular, the modeling effort concentrates on the robustness of the report submission mechanism, a core feature for maintaining data reliability within the platform.

5.1 Signatures

The signatures defined below represent the vocabulary used to describe the BBP ecosystem. Here, we outline the primary atoms of the model, including User, Trip, and Path, while also defining the dynamic elements such as Obstacle and PathStatus that are essential for tracking the system's evolution.

```
enum PathStatus {
    Optimal,
    Medium,
    Sufficient,
    MaintenanceRequired
}

sig User {
    var submittedReports: set PathReport
}

var sig Trip {
    var traveler: one User,
    var path: one Path,
    var emittedReports: lone PathReport
}

sig Path {
    author: one User,
    var reports: set PathReport,
    var currentStatus: one PathStatus,
    var activeObstacles: set Obstacle,
    streets: some Street
}

var sig PathReport {
    var author: one User,
    var reportedStatus: one PathStatus,
    var reportedObstacles: set Obstacle,
} {
    this in Path.reports
}

var sig Obstacle {}

sig Street {} {
    this in Path.streets
}
```

5.2 Facts

The facts represent the constraints that the system must respect and are always true in the model. For the BBP application, we defined numerous facts to enforce data consistency, such as the necessity for a PathReport to be strictly linked to a Trip performed by the author. Furthermore, they dictate the logic for the dynamic evolution of the system, determining how PathStatus and Obstacles are updated based on the aggregation of user reports.

```
//Reports merging operation
fact PathStatusFromReports {
  always (all p: Path |
    one s: PathStatus | {
      p.currentStatus = s
      all s2: PathStatus - s |
        #(p.reports & reportedStatus.s) ≥ #(p.reports & reportedStatus.s2)
      no s_tie: PathStatus - s | //reportedStatus.s2 = { r : PathReport | r.
        ↪ reportedStatus = s2 }
        #(p.reports & reportedStatus.s_tie) = #(p.reports & reportedStatus.s)
        and
        s_tie in s.nexts
    })
}

//Paths must include the same obstacles reported in the reports; if a path receives
↪ multiple reports, it will contain the union of all obstacles from those reports
fact ObstaclesEvolution {
  always (all p: Path |
    (some p.reports) implies {
      p.activeObstacles = p.reports.reportedObstacles
    }
  )
}

//Obstacles must be associated with a Path or a Report; they cannot be unlinked from
↪ everything
fact NoDisconnectedObstacles {
  always (Obstacle in Path.activeObstacles + PathReport.reportedObstacles)
}

//Paths that do not receive any reports in the next step will have the same obstacles as
↪ in the previous step
fact ObstaclesPersistenceWithoutReports {
  always (all p: Path |
    no p.reports' implies
      p.activeObstacles' = p.activeObstacles
    )
}

//report management
fact ReportTripSystemIntegrity {
  always {
    //Each report must be associated with exactly one trip
    all r: PathReport | one t: Trip | t.emittedReports = r

    -- If a trip t has emitted a report r, then:
    -- - The path of the trip must be the same as the reports path
    -- - The traveler must be the author of the report
    all t: Trip | all r: t.emittedReports | {
      t.path = reports.r
      t.traveler = r.author
    }

    //The submittedReports set must contain all and only the reports written by the
    ↪ user
    all u: User | u.submittedReports = {r: PathReport | r.author = u}

    no (Trip & Trip')
  }
}
```

```

//There cannot be multiple paths with the same streets
fact UniquePathTopology {
  all p1, p2: Path | p1.streets = p2.streets  $\implies$  p1 = p2
}

//At any step, a user can make at most one trip
fact OneTripPerUserPerStep {
  always (
    all u: User |
      lone t: Trip | t.traveler = u
  )
}

//If an obstacle is removed from a path, it is removed from all paths that contain it
fact ObstacleDeletionLogic {
  always (all o: Obstacle |
    (o in Path.activeObstacles and o not in Path.activeObstacles') implies {
      (some r: PathReport' | o in r.reportedObstacles) implies o in Obstacle'
      else o not in Obstacle'
    }
  )
}

```

5.3 Assertions

The assertions are the safety properties that we want to verify in the model. These conditions must hold true to ensure the platform avoids invalid or illogical states. For the BBP scenario, we verified several critical aspects, such as the strict requirement that a User must have actually completed a Trip on a specific Path before submitting a report (AuthorWasThere), as well as the constraint that limits users to a single report submission per time step to prevent spam.

```

//Verify that each report must be associated with a trip
assert ReportRequiresTrip {
  always (
    all r: PathReport |
      one t: Trip | r in t.emittedReports
  )
}

//If a path does not receive reports, it will have the same status as the previous step
assert StatusStability {
  always (all p: Path |
    (p.reports' = p.reports)
    implies
    (p.currentStatus' = p.currentStatus)
  )
}

//Verify that at each step, a user can submit at most one report
assert OneReportPerUserPerStep {
  always (
    all u: User |
      #{r: PathReport | r.author = u} <= 1
  )
}

//Verify that each report is associated with exactly one trip
assert ReportBelongsToOnePathOnly {
  always (
    all r: PathReport |
      one p: Path | r in p.reports
  )
}

//Verify that the reports author has completed a trip related to the reported path
assert AuthorWasThere {
  always (
    all p: Path, r: p.reports |
      some t: Trip | t.path = p and t.traveler = r.author
  )
}

```

```

    )
}

//The set of all existing PathReports must be equal to the union of the reports of all
↪ Paths
assert ConservationOfReports {
    always PathReport = Path.reports
}

```

5.4 Model Visualization

The following model was obtained with the following predicate:

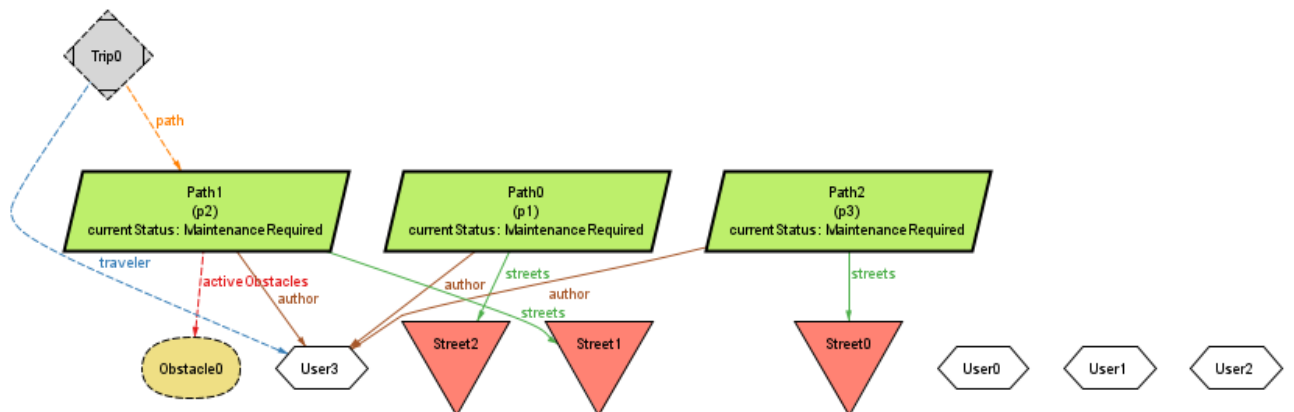
```

pred simulation {
    no PathReport
    #Path = 3
    #Street = 3
    #User = 4
    after {
        some disj p1, p2, p3: Path {
            #p1.reports = 2
            all r1, r2: p1.reports | r1 ≠ r2 implies r1.reportedStatus ≠ r2.
            ↪ reportedStatus
            #p2.reports = 1
            #p3.reports = 1
        }
        #Obstacle = 4
        no (PathReport & PathReport')
    }
    after after {
        #Obstacle = 2
        #Trip = 4
        one p: Path | {
            p.reports = PathReport
            #{r: p.reports | r.reportedStatus = Medium} = 2
            #{r: p.reports | r.reportedStatus = Sufficient} = 1
        }
    }
}

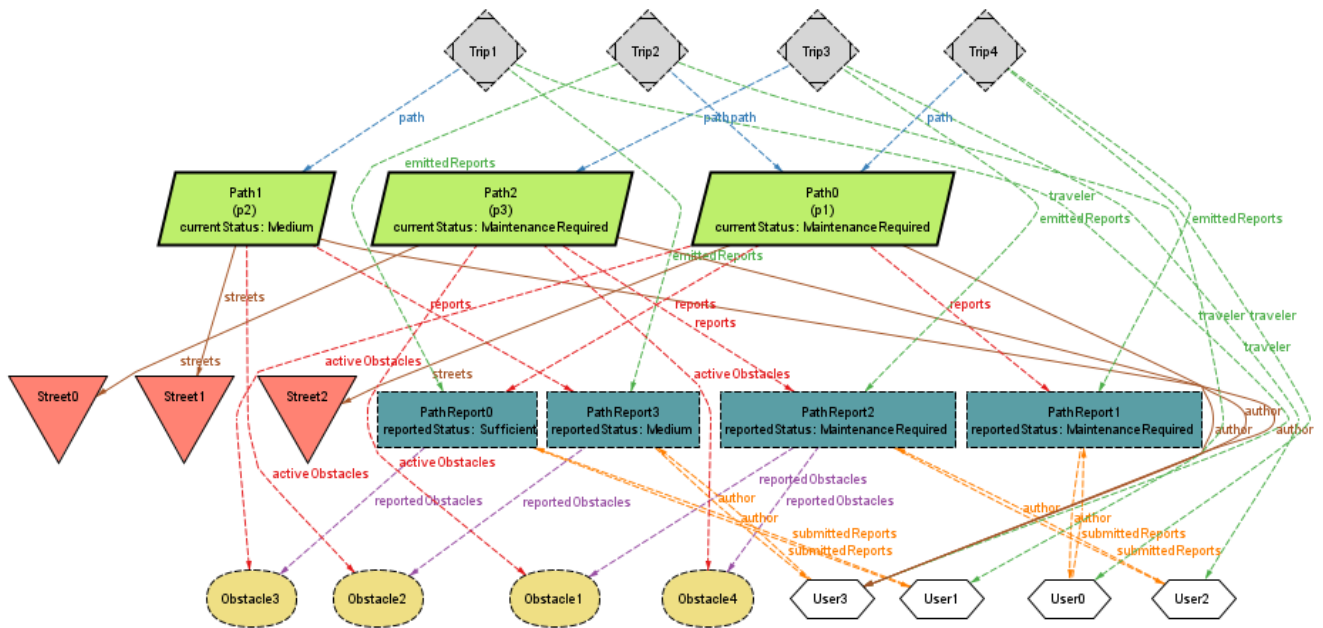
run simulation for 10

```

Timeframe 0 (focus on discard trip):



User3 takes a trip on Path1 but decides to discard it; therefore, no report is submitted. The status of all paths remains unchanged, and the set of obstacles remains the same.

Timeframe 1 (focus on tie and add/remove obstacles):

User0 emits a report on Path0 with status = 'Manteinance Required'.

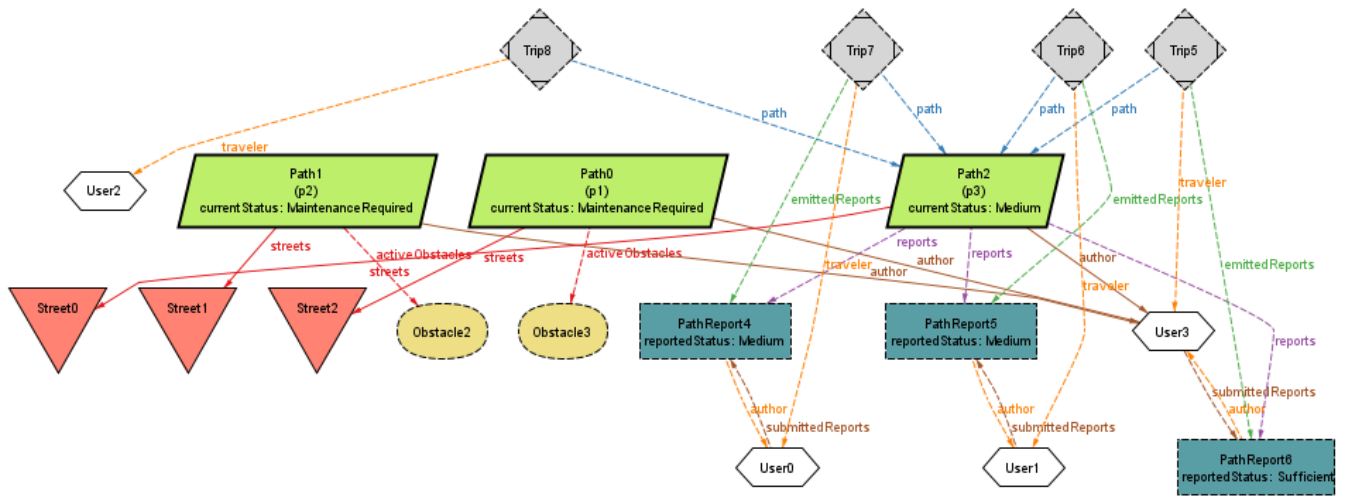
User1 emits a report on Path0 with status = 'Sufficient' and one obstacle (Obstacle3).

User2 emits a report on Path2 with status = 'Manteinance Required' and two obstacles (Obstacle1 and Obstacle4).

User3 emits a report on Path1 with status = 'Medium' and one obstacle (Obstacle 2), and it removes Obstacle0.

As a result, Path1 and Path2 take the status of the single report associated, while since there is a tie in the count of report statuses for Path0, it is assigned the lower of the two statuses (Maintenance Required).

In addition, Obstacle0 will be replaced with Obstacle3 in Path1, Obstacle1 and Obstacle4 have been added to Path2, and Obstacle2 has been added to Path1.

Timeframe 2 (focus on merging):

User0 emits a report on Path2 with status = 'Medium'.

User1 emits a report on Path2 with status = 'Medium'.

User3 emits a report on Path2 with status = 'Sufficient'.

User2 makes a trip on Path2 but decides to discard it, therefore no report is submitted.

As a result, the status of Path2 is updated to Medium according to the majority vote, while the remaining paths are unaffected.

6 Use of Generative AI

6.1 AI-Based Support Tools

While tools such as ChatGPT were utilized transversally to support drafting, code refinement, and logical requirement analysis, the specific phase of User Interface Prototyping relied on models specialized in image generation.

Specifically, Google AI Studio was employed to interact with the Gemini model. Thanks to its multi-modal capabilities, it was possible to translate the textual descriptions of Use Cases into high-fidelity visual mockups. This approach allowed for a rapid visualization of the spatial organization of elements (layout).

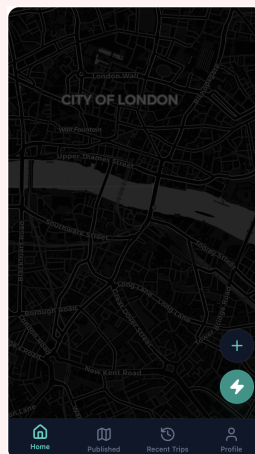
6.2 Example of Interaction: Mockup Generation

A significant example of this interaction is represented by the definition of the Homepage. Initially, a prompt was provided to the model aimed at establishing the general structure and color palette, inspired by leading market competitors.

Prompt Generate a mockup for the homepage of a mobile GPS navigation application. The interface must be in "Dark Mode", featuring a dark night-time street map background similar to Google Maps. At the bottom of the screen, include a navigation bar with four sections: "Home", "Published", "Recent Trip", and "Profile". Position two round floating buttons stacked vertically immediately above this bar in the bottom right corner: one for "Manual Mode" and the other for "Automated Mode".

Gemini Here is a proposal for the Homepage design in night mode. The generated image presents:

- A dark-toned cartographic background (midnight blue and charcoal) with highlighted roads.
- A bottom navigation bar featuring the four requested sections.
- The two action buttons ("Manual" and "Automated") positioned at the bottom right for easy thumb access.



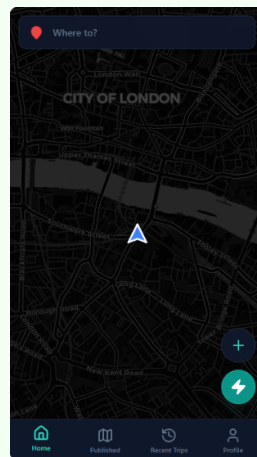
Although the structure was correct, the first iteration lacked crucial elements for immediate usability, such as a clear indication of the user's position and the destination input field. A second refinement prompt was therefore necessary to integrate these details without altering the already approved visual style.

Prompt Based on the image just generated, modify only the top area and the center of the map. Add a floating search bar at the top of the screen specifically for entering a destination (with a placeholder like "Where to?"). Additionally, place a prominent cursor, such as a glowing blue arrow, in the center of the map to indicate the user's current position. Ensure that the rest of the existing interface, including the bottom navigation bar, remains unchanged.

Gemini Updated the mockup by integrating the requested elements:

- **Search Bar:** Inserted at the top with rounded corners and a search icon, maintaining the transparency typical of modern UIs.
- **Position Cursor:** Added a blue indicator with a slight glow effect at the center of the map to ensure immediate contrast against the dark background.

The rest of the layout (navbar and mode buttons) remained unchanged.



This two-phase process allowed for the separation of the structural layout definition (navbar, map, buttons) from the functional details (search bar, cursor). The model demonstrated the ability to maintain stylistic consistency between generations, integrating the requested modifications naturally within the pre-existing visual context.

6.3 Example of Interaction: Rules Identification

The following interaction describes a consultation aimed at defining the Standard Compliance section. We engaged the AI to clarify the complex requirements related to the handling of continuous geolocation history and the integration of external map services. Through two prompts, we progressed from identifying the applicable external regulations (GDPR, Terms of Service) to defining the internal technical solutions required to ensure compliance without compromising the platform's functionality.

Prompt I am developing a mobile app for cyclists that features real-time navigation and trip tracking, relying on third-party map services. Considering I will be handling continuous geolocation data, what are the primary privacy regulations and map licensing constraints I need to strictly follow?

ChatGPT Based on your description, the platform operates at the intersection of sensitive user data and proprietary services. Here are the mandatory constraints:

1. Privacy and Sensitive Data: Since the system handles Personally Identifiable Information (PII) and continuous geolocation history, this constitutes a significant privacy risk as it reveals user habits and private locations. You must strictly comply with the General Data Protection Regulation (GDPR) and potentially the California Consumer Privacy Act (CCPA).
2. Map Provider Terms of Service (ToS): The dependency on external providers (e.g., OpenStreetMap or Google Maps) mandates rigorous adherence to their ToS. You are required to:
 - Dynamically display proper copyright attributions.
 - Strictly enforce technical limitations on offline data caching to avoid licensing violations.

The analysis of the regulatory landscape revealed a critical tension: the application requires continuous data collection for functionality, yet strictly limits data retention for compliance. Recognizing that simple adherence to rules is insufficient, we directed the next query toward defining the architectural drivers necessary to resolve this conflict at the structural level.

Prompt Based on these regulatory constraints, how should I design the software architecture to ensure compliance? Specifically, I need to understand how to embed "Privacy-by-Design" principles into the core system and how to technically enforce the caching limitations, while keeping the architecture flexible enough to adapt to future changes.

ChatGPT To ensure compliance without compromising functionality, the system must be architected with specific drivers:

- **Privacy-by-Design Core:** You must embed privacy-by-design principles directly into the software core, rather than treating compliance as an external add-on. This minimizes the risk of exposing user habits.
- **Adaptability and Flexibility:** The system must be architected with the flexibility to adapt to this dual set of regulatory and licensing constraints. Regulations and ToS change frequently; a modular design will allow you to update compliance logic without rewriting the entire application.
- **Technical Enforcement:** The architecture must include a logic layer that automatically manages data retention, ensuring you strictly enforce technical limitations on offline data caching as required by the map providers.

The rest of the layout (navbar and mode buttons) remained unchanged.

Verification

We verified the information generated by the AI by checking official online sources. The details regarding privacy regulations and map service policies were confirmed to be accurate and consistent with real-world requirements. Therefore, the content is reliable and has been validated for use in this project.

6.4 Conclusion

The integration of AI-based tools has significantly improved efficiency in the preparation of complex technical documents. Nevertheless, despite their advantages, it remains essential to adopt a critical approach and carefully review the produced content to ensure accuracy, coherence, and completeness. Achieving high-quality results often requires multiple iterations and precise refinements, guiding the process step by step. AI tools have proven to be valuable support instruments, but they should be used with careful supervision throughout all phases of document development.

7 Effort Spent

Andrea Ferritti

Section	Hours
Introduction	12
Overall Description	15
Specific Requirements	23
Formal Analysis Using Alloy	16
Use of Generative AI	2

Table 22: Effort - Andrea Ferritti

Jieke Lin

Section	Hours
Introduction	10
Overall Description	13
Specific Requirements	26
Formal Analysis Using Alloy	17
Use of Generative AI	2

Table 23: Effort - Jieke Lin

Tommaso Tosca

Section	Hours
Introduction	9
Overall Description	15
Specific Requirements	21
Formal Analysis Using Alloy	24
Use of Generative AI	2

Table 24: Effort - Tommaso Tosca